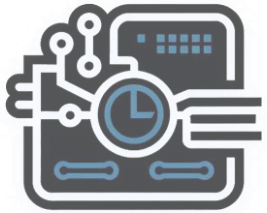


Progettazione Elettronica Digitale

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

L06 – Sommario

➔Ottimizzazione di FSM
➔Sistemi di controllo complesso



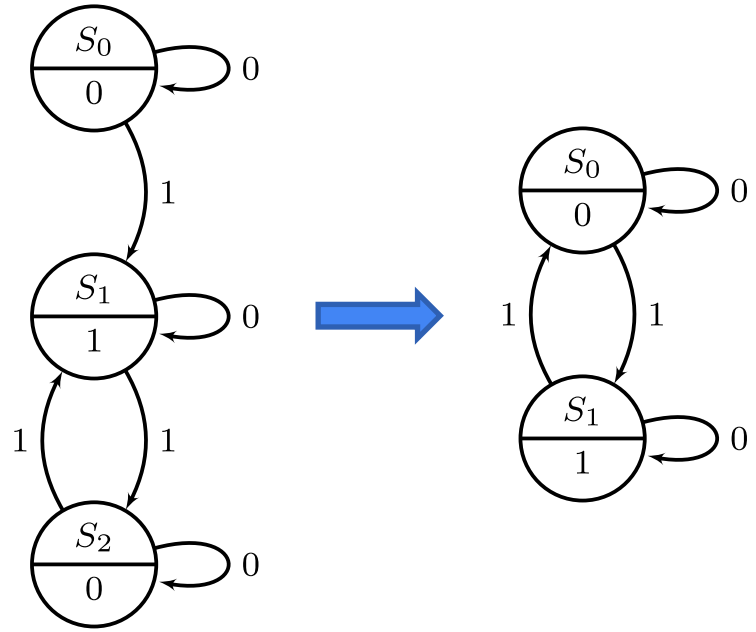
Minimizzazione degli stati

- Ha come obiettivo *identificare e combinare stati* che presentino lo stesso *comportamento*
- Stati *equivalenti*
  - Per *qualsiasi ingresso* producono le *stesse uscite*
  - Hanno *transizioni* verso stati *uguali* o *equivalenti*
- Motivazione
  - Una FSM minimizzata richiede solitamente *meno flip-flop*
  - Meno stati di solito portano a *più don't care*
  - Solitamente richiede *meno porte logiche* per la realizzazione
  - I *limiti più rilevanti* sono dati dalle potenze di 2
    - 1 bit → 2 stati, 2 bit → 4 stati, 3 bit → 8 stati, e così via…

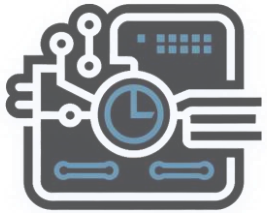
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio: controllore di parità

- Analizziamo *due alternative* con lo *stesso comportamento in uscita* per *qualsiasi sequenza di ingresso*
- FSM *equivalenti*, ma con *diversa realizzazione*
  - Es: diverso numero di stati, o temporizzazione della macchina
- L'approccio classico prevede prima la *fase di definizione degli stati*, e a seguire i *passi di minimizzazione*
- S_0 ed S_2 nell'esempio
  - Producono entrambi l'uscita 0
  - Passano a S_1 con ingresso a 1 e tornano su loro stessi con ingresso a 0



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



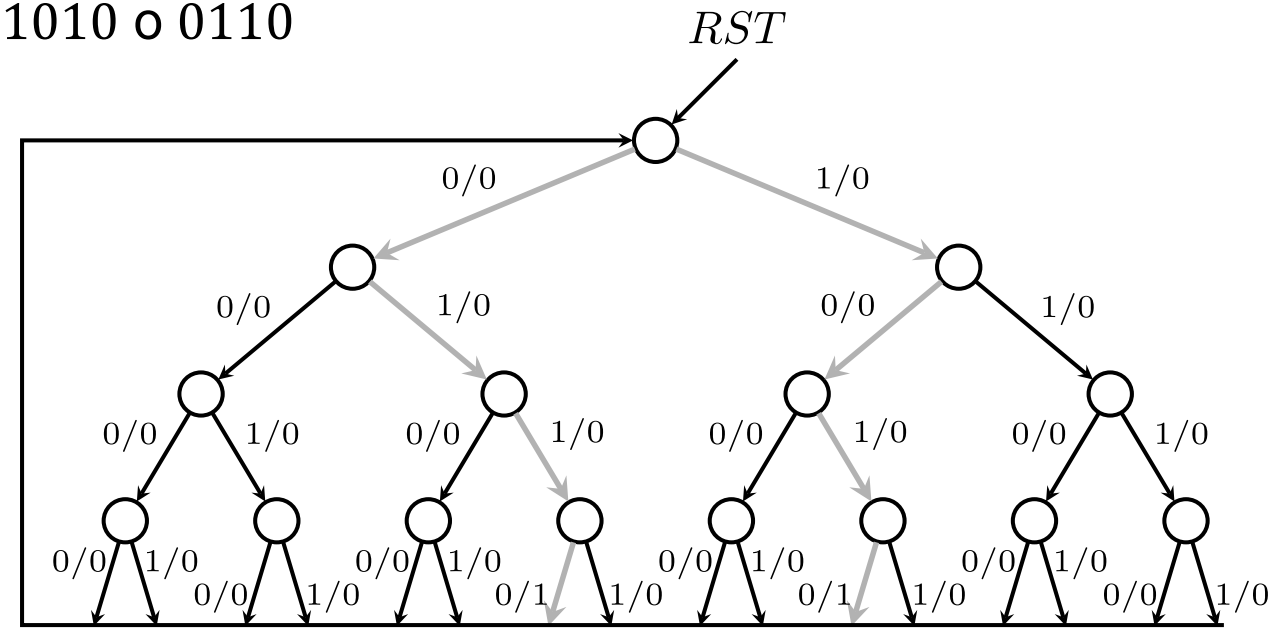
Minimizzazione degli stati
Concetti base

- Si parte dalla *tabella delle transizioni* tra gli stati
- Si identificano stati con lo
  stesso comportamento per quanto riguardi le *uscite*
- Se gli stati hanno lo
  stesso insieme di stati futuri,
  possono essere dichiarati ***equivalenti***
- Si *combinano* insieme *coppie di stati equivalenti*
- Si *ripetono* tutti i passi precedenti fino a quando
  non si trovano più altre coppie di stati equivalenti

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della corrispondenza tra righe

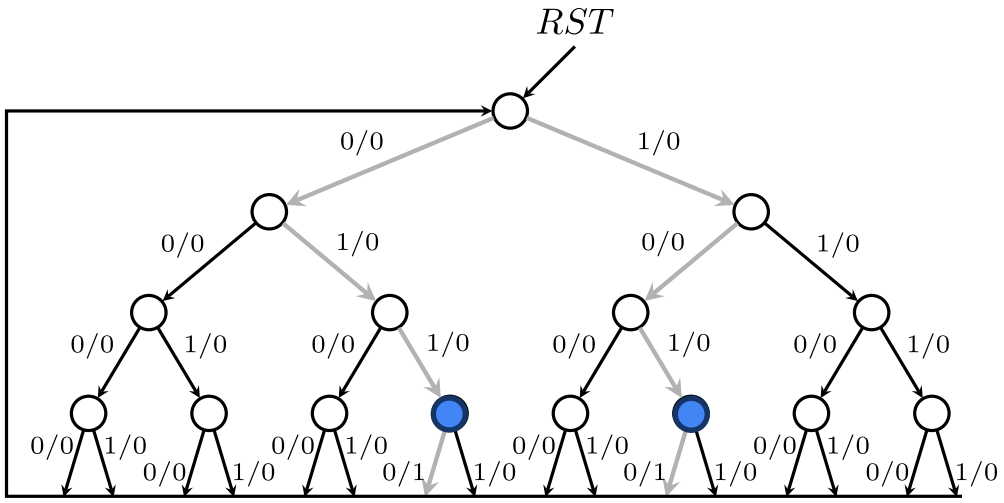
- Esempio di FSM di riferimento
  - *Un ingresso* e *una uscita*
  - Per *sequenze di ingresso a gruppi di* 4 *bit*, produce 1 in uscita se l'ultimo gruppo è 1010 o 0110
    → I *cammini in grigio* nell'albero binario
- Esempi di comportamento
  - Ingresso: $0010\ 0110\ 1100\ 1010\ 0011\ \ldots$
  - Uscita: $0000\ 0001\ 0000\ 0001\ 0000\ \ldots$
- Limiti superiori per la FSM
  - 15 stati $(1 + 2 + 4 + 8)$
  - 30 transizioni $(2 + 4 + 8 + 16)$
  → Sono sufficienti a riconoscere *qualsiasi sequenza di* 4 *bit*



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della corrispondenza tra righe

- Si applica iterativamente una fase di *esplorazione della tabella degli stati* in cui si identificano *corrispondenze tra righe* della stessa
  - $S_{10} \equiv S_{12}$

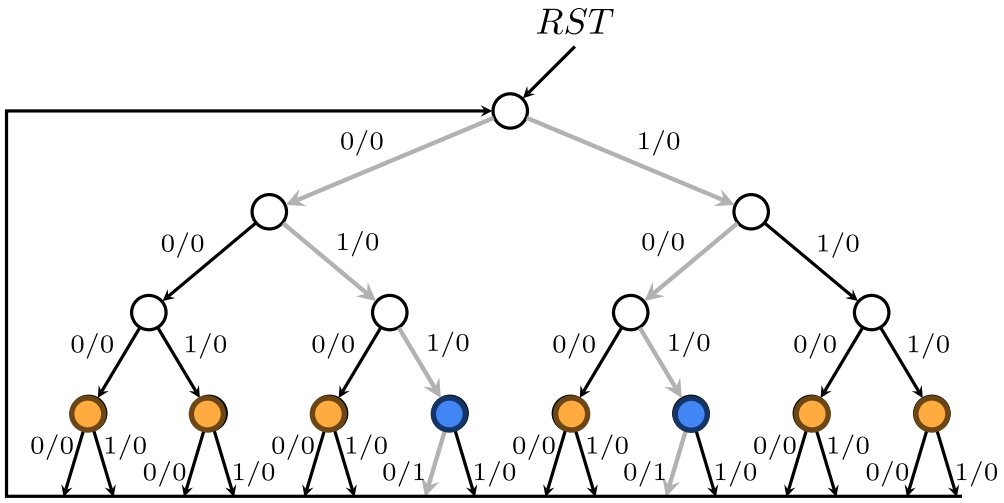


Sequenza Ingresso	Stato Presente	Stato Futuro $x=0$	Stato Futuro $x=1$	Uscita $x=0$	Uscita $x=1$
RST	S_0	S_1	S_2	0	0
0	S_1	S_3	S_4	0	0
1	S_2	S_5	S_6	0	0
00	S_3	S_7	S_8	0	0
01	S_4	S_9	S_{10}	0	0
10	S_5	S_{11}	S_{12}	0	0
11	S_6	S_{13}	S_{14}	0	0
000	S_7	S_0	S_0	0	0
001	S_8	S_0	S_0	0	0
010	S_9	S_0	S_0	0	0
011	S_{10}	S_0	S_0	1	0
100	S_{11}	S_0	S_0	0	0
101	S_{12}	S_0	S_0	1	0
110	S_{13}	S_0	S_0	0	0
111	S_{14}	S_0	S_0	0	0

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della corrispondenza tra righe

- Si applica iterativamente una fase di *esplorazione della tabella degli stati* in cui si identificano *corrispondenze tra righe* della stessa
  - $S_{10} \equiv S_{12} \rightarrow S'_{10}$
  - $S_7 \equiv S_8 \equiv S_9 \equiv S_{11} \equiv S_{13} \equiv S_{14}$

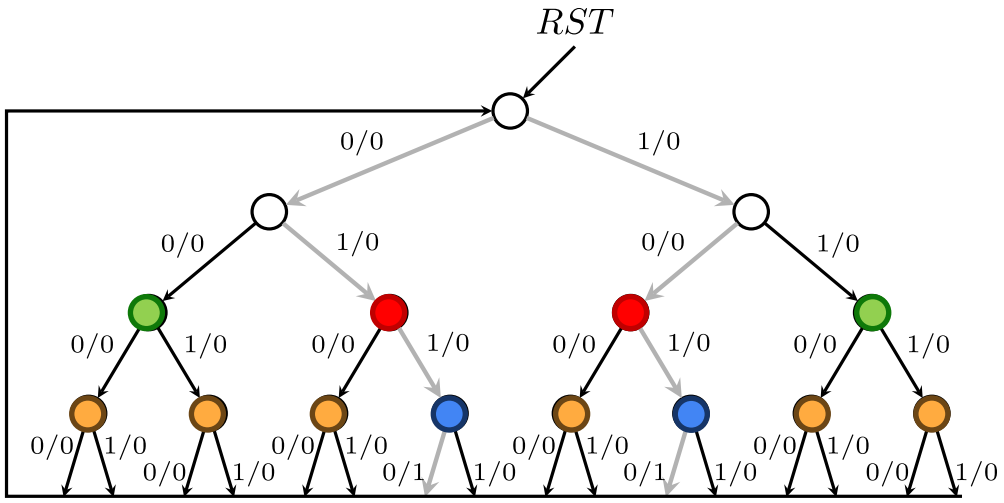


Sequenza Ingresso	Stato Presente	Stato Futuro $x=0$	Stato Futuro $x=1$	Uscita $x=0$	Uscita $x=1$
RST	S_0	S_1	S_2	0	0
0	S_1	S_3	S_4	0	0
1	S_2	S_5	S_6	0	0
00	S_3	S_7	S_8	0	0
01	S_4	S_9	S'_{10}	0	0
10	S_5	S_{11}	S'_{10}	0	0
11	S_6	S_{13}	S_{14}	0	0
000	S_7	S_0	S_0	0	0
001	S_8	S_0	S_0	0	0
010	S_9	S_0	S_0	0	0
011 | 101	S'_{10}	S_0	S_0	1	0
100	S_{11}	S_0	S_0	0	0
110	S_{13}	S_0	S_0	0	0
111	S_{14}	S_0	S_0	0	0

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della corrispondenza tra righe

- Si applica iterativamente una fase di *esplorazione della tabella degli stati* in cui si identificano *corrispondenze tra righe* della stessa
  - $S_{10} \equiv S_{12} \rightarrow S'_{10}$
  - $S_7 \equiv S_8 \equiv S_9 \equiv S_{11} \equiv S_{13} \equiv S_{14} \rightarrow S'_7$
  - $S_3 \equiv S_6$
  - $S_4 \equiv S_5$

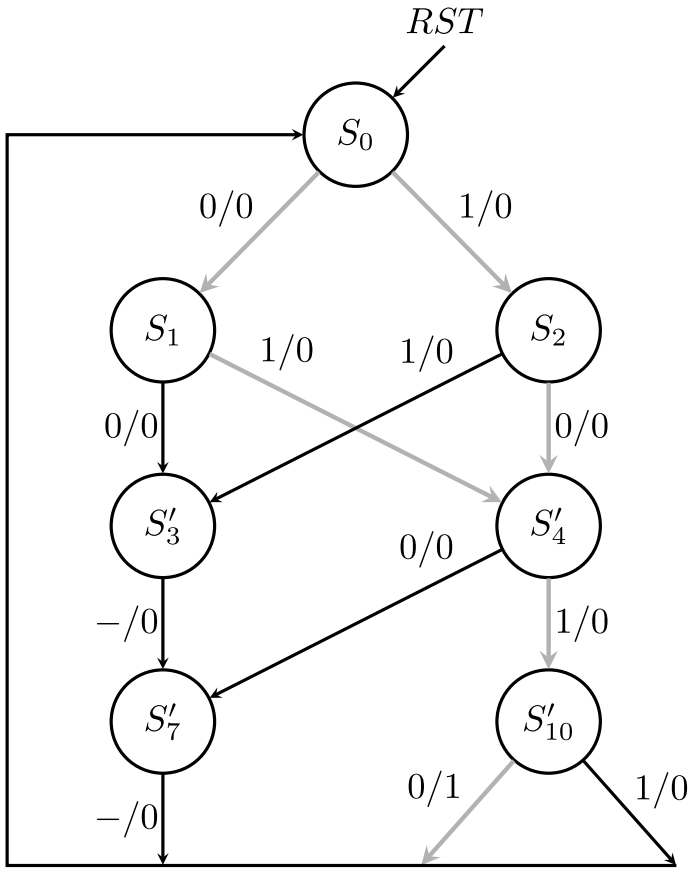


Sequenza Ingresso	Stato Presente	Stato Futuro $x=0$	Stato Futuro $x=1$	Uscita $x=0$	Uscita $x=1$
RST	S_0	S_1	S_2	0	0
0	S_1	S_3	S_4	0	0
1	S_2	S_5	S_6	0	0
00	S_3	S'_7	S'_7	0	0
01	S_4	S'_7	S'_{10}	0	0
10	S_5	S'_7	S'_{10}	0	0
11	S_6	S'_7	S'_7	0	0
$\neg(011 \mid 101)$	S'_7	S_0	S_0	0	0
$011 \mid 101$	S'_{10}	S_0	S_0	1	0

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della corrispondenza tra righe

- Quando si raggiunge un *fix-point*, la tabella degli stati è *minimizzata*
  → Se ne può derivare il corrispondente *diagramma degli stati ridotto*

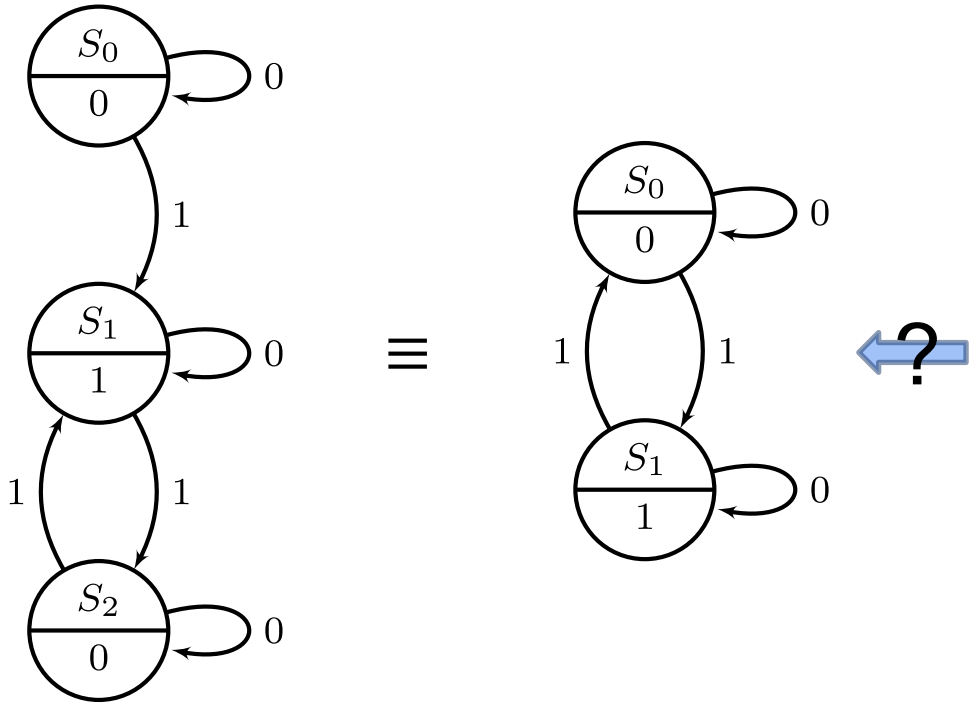


Sequenza Ingresso	Stato Presente	Stato Futuro $x=0$	Stato Futuro $x=1$	Uscita $x=0$	Uscita $x=1$
RST	S_0	S_1	S_2	0	0
0	S_1	S'_3	S'_4	0	0
1	S_2	S'_4	S'_3	0	0
00 | 11	S'_3	S'_7	S'_7	0	0
01 | 10	S'_4	S'_7	S'_{10}	0	0
$\neg(011 \mid 101)$	S'_7	S_0	S_0	0	0
011 | 101	S'_{10}	S_0	S_0	1	0

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della corrispondenza tra righe

- Il metodo della corrispondenza tra righe è facile da capire e realizzare
- Ma non sempre conduce al *numero minimo di stati*, anche in casi molto semplici
  - Esempio: nel controllore di parità a 3 stati
    S_0 ed S_2 *non* si possono riconoscere come equivalenti usando il *criterio dello stato futuro*

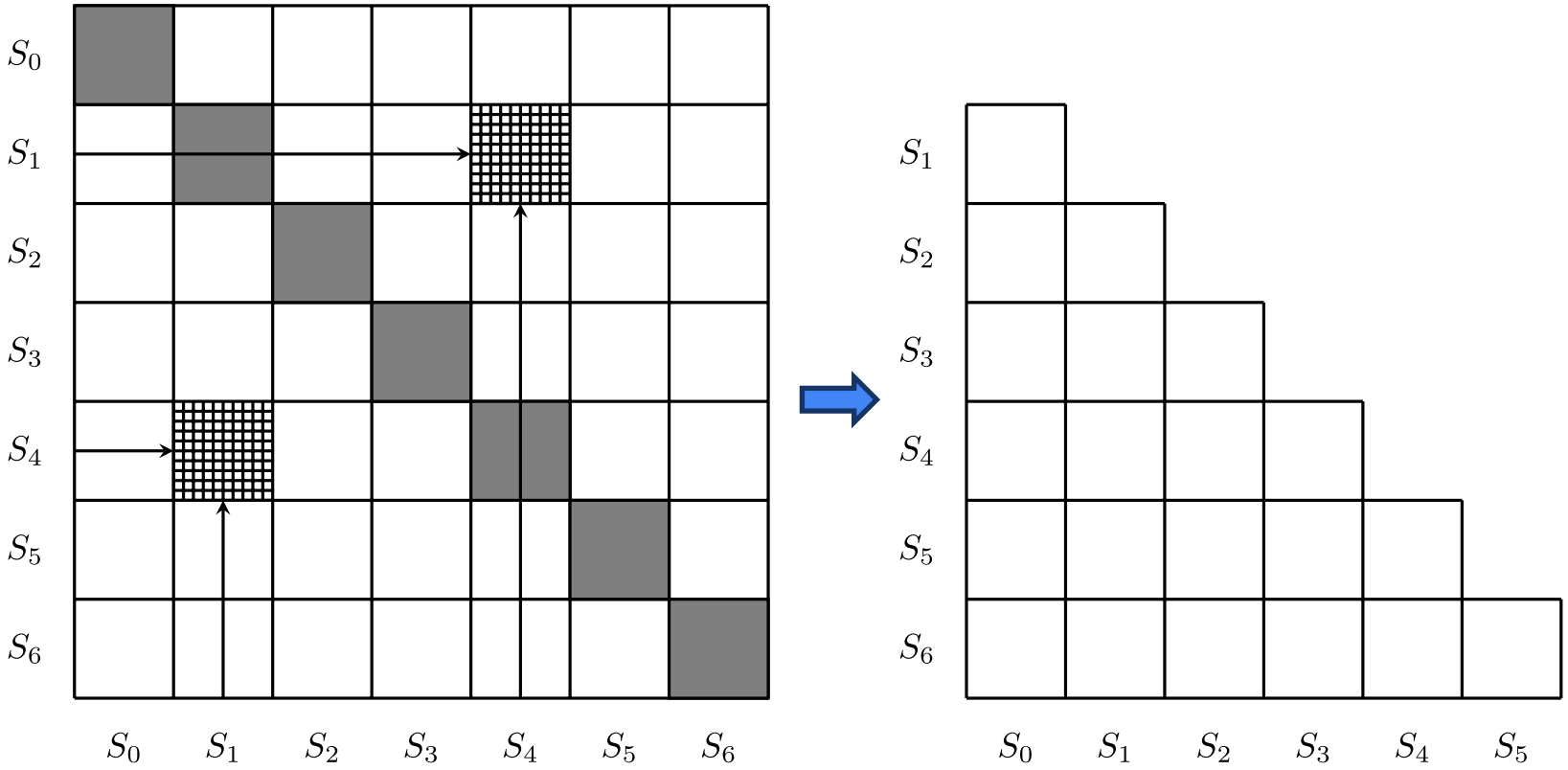


Stato Presente	Stato Futuro $x=0$	Stato Futuro $x=1$	Uscita
S_0	S_0	S_1	0
S_1	S_1	S_2	1
S_2	S_2	S_1	0

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della tabella delle implicazioni
Scopo, proprietà

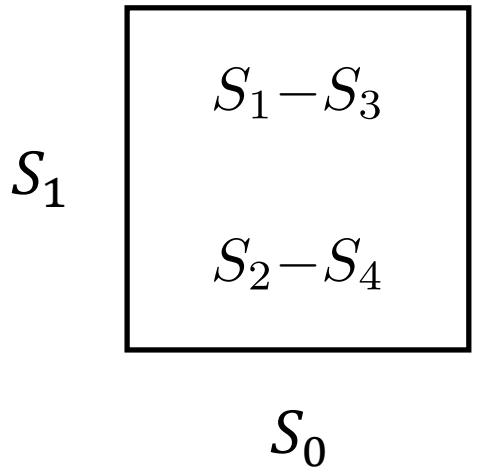
- Tabella delle implicazioni
  - *Enumera esplicitamente* tutte le possibili *coppie di stati*
  - Per *ogni coppia* si valutano gli *stati futuri* per *ogni combinazione d'ingresso*
- *Simmetrica* (es: $S_1 \equiv S_4 \leftrightarrow S_4 \equiv S_1$) e *riflessiva* (es: $S_1 \equiv S_1$) rispetto alla diagonale



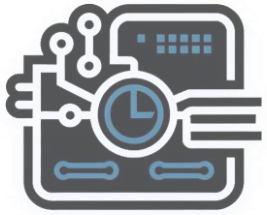
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della tabella delle implicazioni
Costruzione

- Tecnica di riempimento della tabella: una cella (i, j)
  - È associata alla coppia di stati (S_i, S_j)
  - Riporta le *coppie di stati futuri* raggiunte da S_i ed S_j per *ciascun valore d'ingresso*
- Due stati S_i ed S_j possono essere *equivalenti* solo se
  1. Hanno le *stesse uscite* → se hanno *uscite diverse*, la cella (i, j) viene *subito cancellata*
  2. *Tutte le coppie di stati futuri* riportate nella cella sono a loro volta *equivalenti*
- Esempio
  - S_0 va in S_1 con $x = 0$ e in S_2 con $x = 1$
  - S_1 va in S_3 con $x = 0$ e in S_4 con $x = 1$
  - Nella cella $(1, 0)$ si riportano quindi le *implicazioni*
    - $(S_1 - S_3)$ per $x = 0$
    - $(S_2 - S_4)$ per $x = 1$



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Metodo della tabella delle implicazioni
Esempio di riferimento

- FSM di riferimento
  - Un ingresso e una uscita
  - Produce 1 in uscita se la sequenza più recente di 3 bit è 010 oppure 110

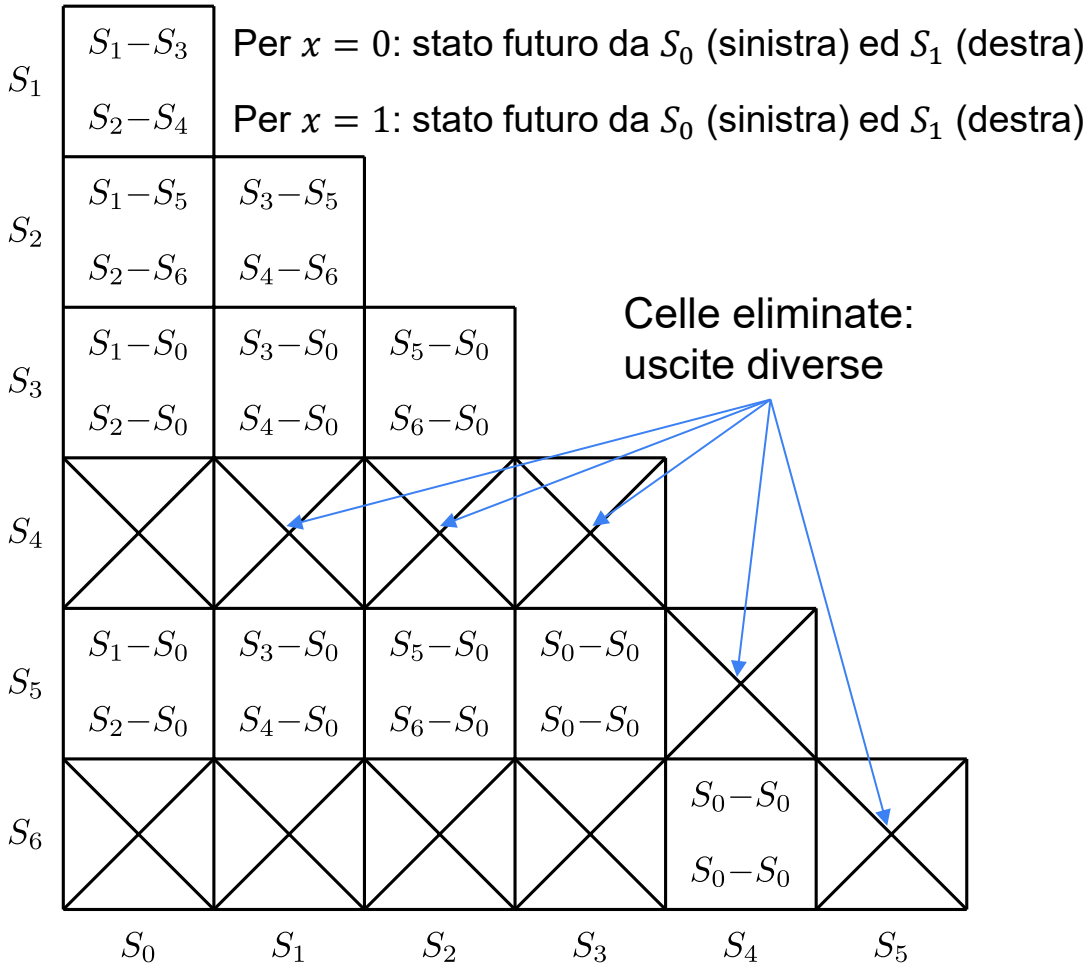
		Stato Futuro		Uscita	
Sequenza Ingresso	Stato Presente	$x=0$	$x=1$	$x=0$	$x=1$
RST	S_0	S_1	S_2	0	0
0	S_1	S_3	S_4	0	0
1	S_2	S_5	S_6	0	0
00	S_3	S_0	S_0	0	0
01	S_4	S_0	S_0	1	0
10	S_5	S_0	S_0	0	0
11	S_6	S_0	S_0	1	0

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della tabella delle implicazioni
Controllo compatibilità uscite

		Stato Futuro		Uscita	
Sequenza Ingresso	Stato Presente	$x=0$	$x=1$	$x=0$	$x=1$
RST	S_0	S_1	S_2	0	0
0	S_1	S_3	S_4	0	0
1	S_2	S_5	S_6	0	0
00	S_3	S_0	S_0	0	0
01	S_4	S_0	S_0	1	0
10	S_5	S_0	S_0	0	0
11	S_6	S_0	S_0	1	0

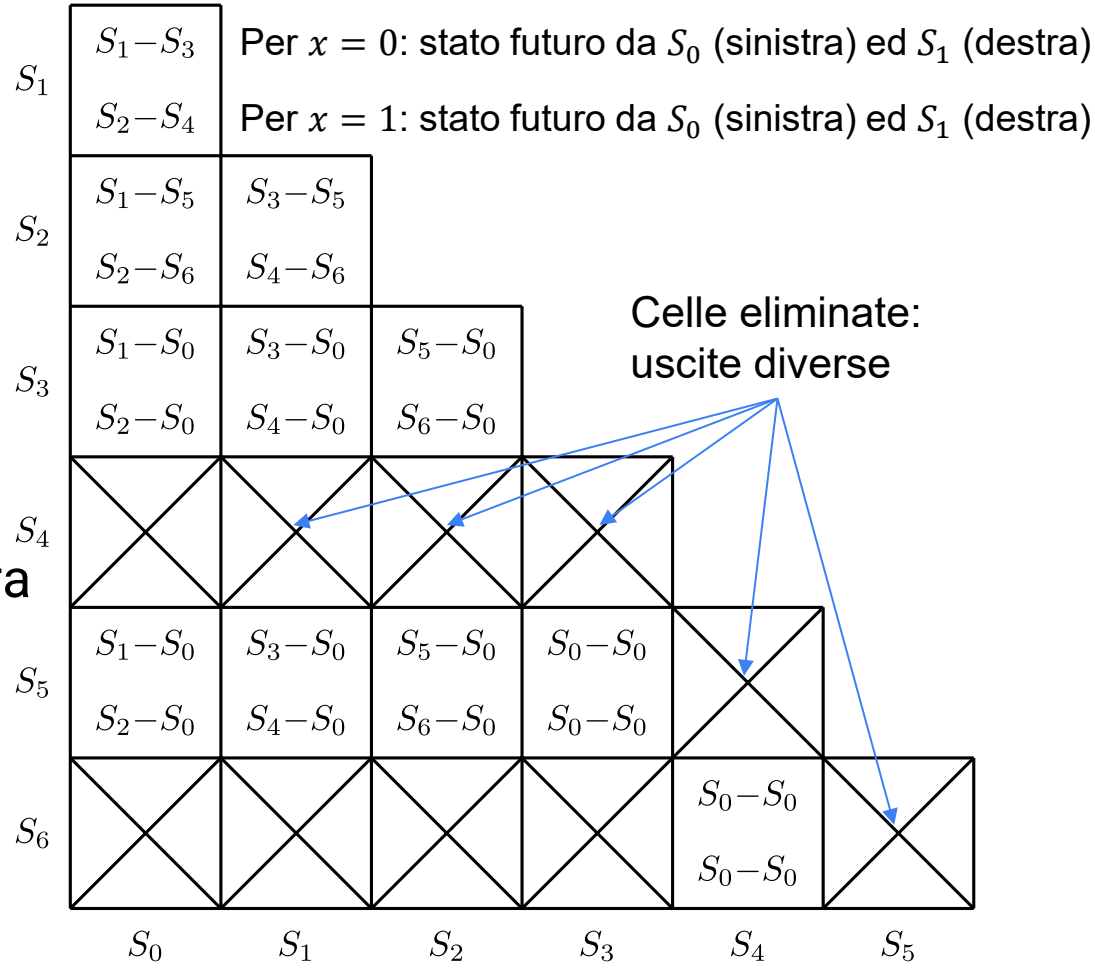
- Ogni cella contiene, per ciascun ingresso, la *coppia di stati futuri da confrontare*
- Le celle corrispondenti a coppie di stati con *uscite diverse* vengono *cancellate subito*
- Le celle rimanenti esprimono *implicazioni da verificare* nelle iterazioni successive



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della tabella delle implicazioni
Propagazione delle incompatibilità

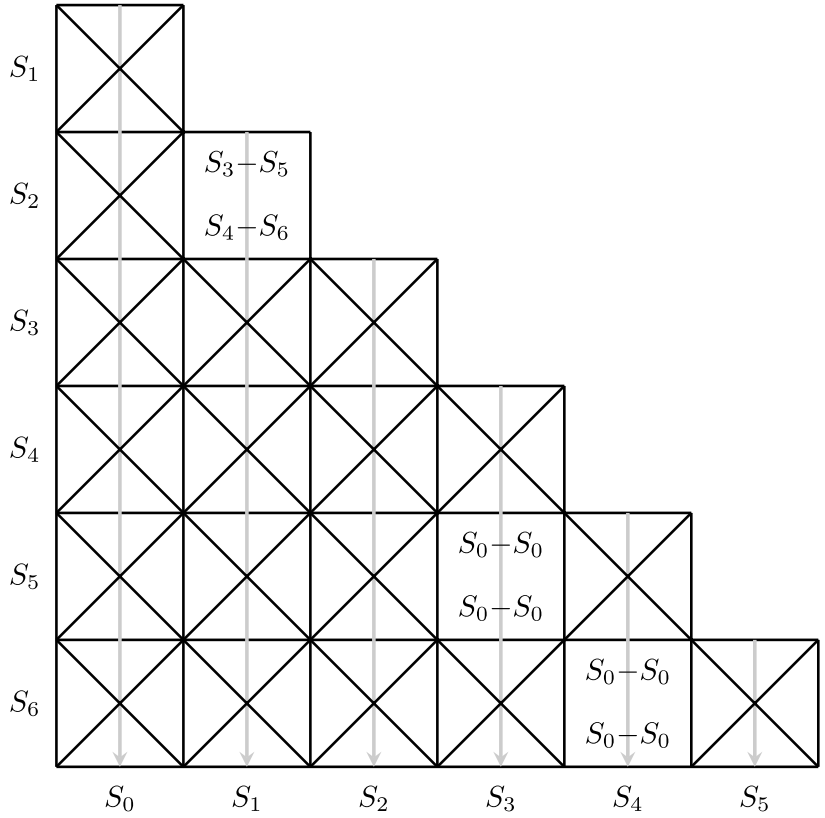
- La tabella delle implicazioni è la *traduzione in forma tabellare* della *tabella di transizione*
- Le *equivalenze* si determinano *esplorando iterativamente* le *implicazioni* riportate nelle celle
  - Scansione tipica:
    dall'alto verso il basso e da sinistra verso destra
- Esempio
  - S_2 ed S_4 hanno *uscite diverse*
    → La cella $(4,2)$ viene *cancellata*
  - Di conseguenza, anche S_0 ed S_1 non possono essere equivalenti, perché la cella $(1,0)$ implica la coppia (S_2, S_4) per $x = 1$



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della tabella delle implicazioni
Marcatura delle equivalenze fino al fix-point

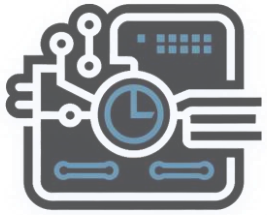
- *Primo passo* di marcatura
  - $S_3 \equiv S_5 \rightarrow S_3'$
  - $S_4 \equiv S_6 \rightarrow S_4'$
- *Secondo passo* di marcatura
  - La cella (S_1, S_2) non contiene più *implicazioni incompatibili*, quindi $S_1 \equiv S_2 \rightarrow S_1'$
- Le iterazioni successive non producono altre *cancellazioni* o nuove *equivalenze*
  → È stato raggiunto il *fix-point*



Sequenza Ingresso	Stato Presente	Stato Futuro $x=0$	Stato Futuro $x=1$	Uscita $x=0$	Uscita $x=1$
RST	S_0	S_1	S_2	0	0
0	S_1	S_3	S_4	0	0
1	S_2	S_5	S_6	0	0
00	S_3	S_0	S_0	0	0
01	S_4	S_0	S_0	1	0
10	S_5	S_0	S_0	0	0
11	S_6	S_0	S_0	1	0

Sequenza Ingresso	Stato Presente	Stato Futuro $x=0$	Stato Futuro $x=1$	Uscita $x=0$	Uscita $x=1$
RST	S_0	S_1'	S_1'	0	0
0 | 1	S_1'	S_3'	S_4'	0	0
00 | 10	S_3'	S_0	S_0	0	0
01 | 11	S_4'	S_0	S_0	1	0

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



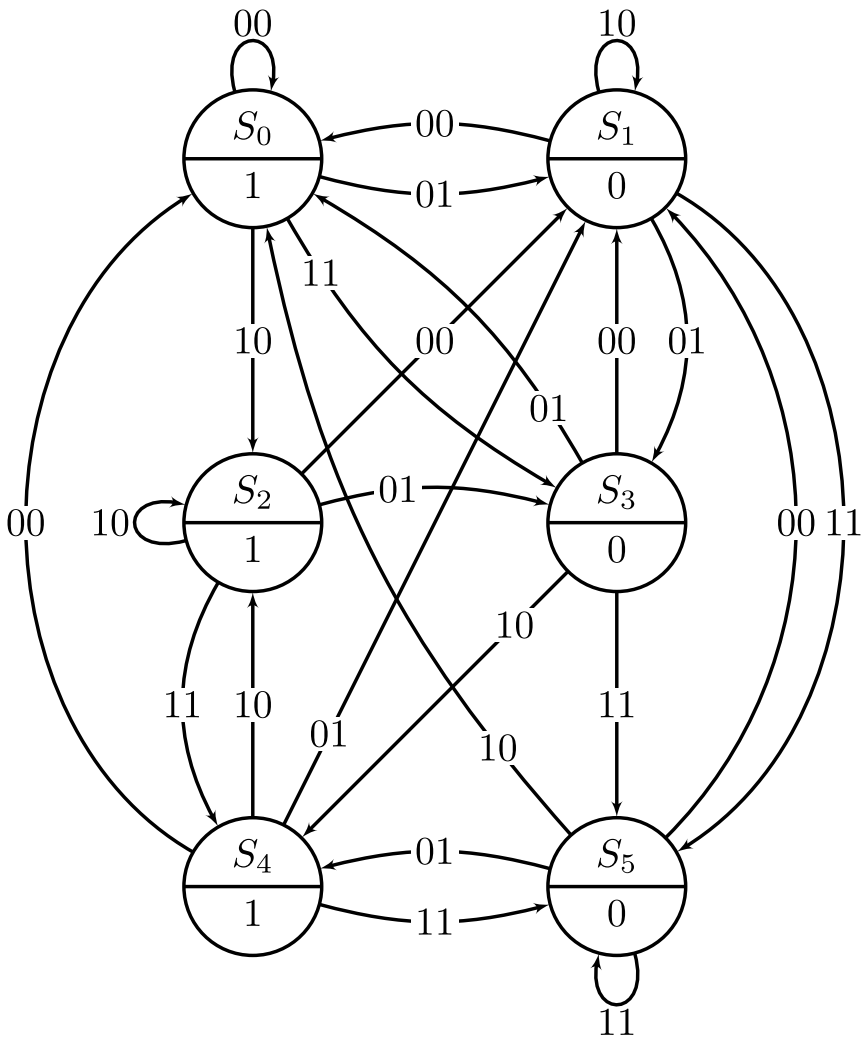
Metodo della tabella delle implicazioni
Algoritmo

- Costruire la tabella delle implicazioni, con una cella per ogni coppia di stati
- *Omettere* la porzione *triangolare superiore* della tabella e la *diagonale* per le proprietà di *simmetria e riflessività*, rispettivamente
- *Cancellare* tutte le celle corrispondenti a coppie di stati con *uscite differenti*
- Completare le *celle rimanenti* con le *implicazioni* associate a *ogni combinazione di ingresso*
- Scorrere la tabella dall'alto verso il basso e da sinistra a destra
  - Se la cella (i, j) contiene almeno una coppia $(S_m - S_n)$ la cui cella corrispondente è cancellata, cancellare anche (i, j)
  - *Ripetere iterativamente* questo passo fino a quando non è più possibile cancellare altre celle
- Al termine, ogni coppia (S_i, S_j) corrispondente a una cella (i, j) *non cancellata* è una *coppia di stati equivalenti*

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della tabella delle implicazioni
Esempio con due variabili d'ingresso

- FSM con *due ingressi* e una *uscita*, rappresentata come macchina di Moore

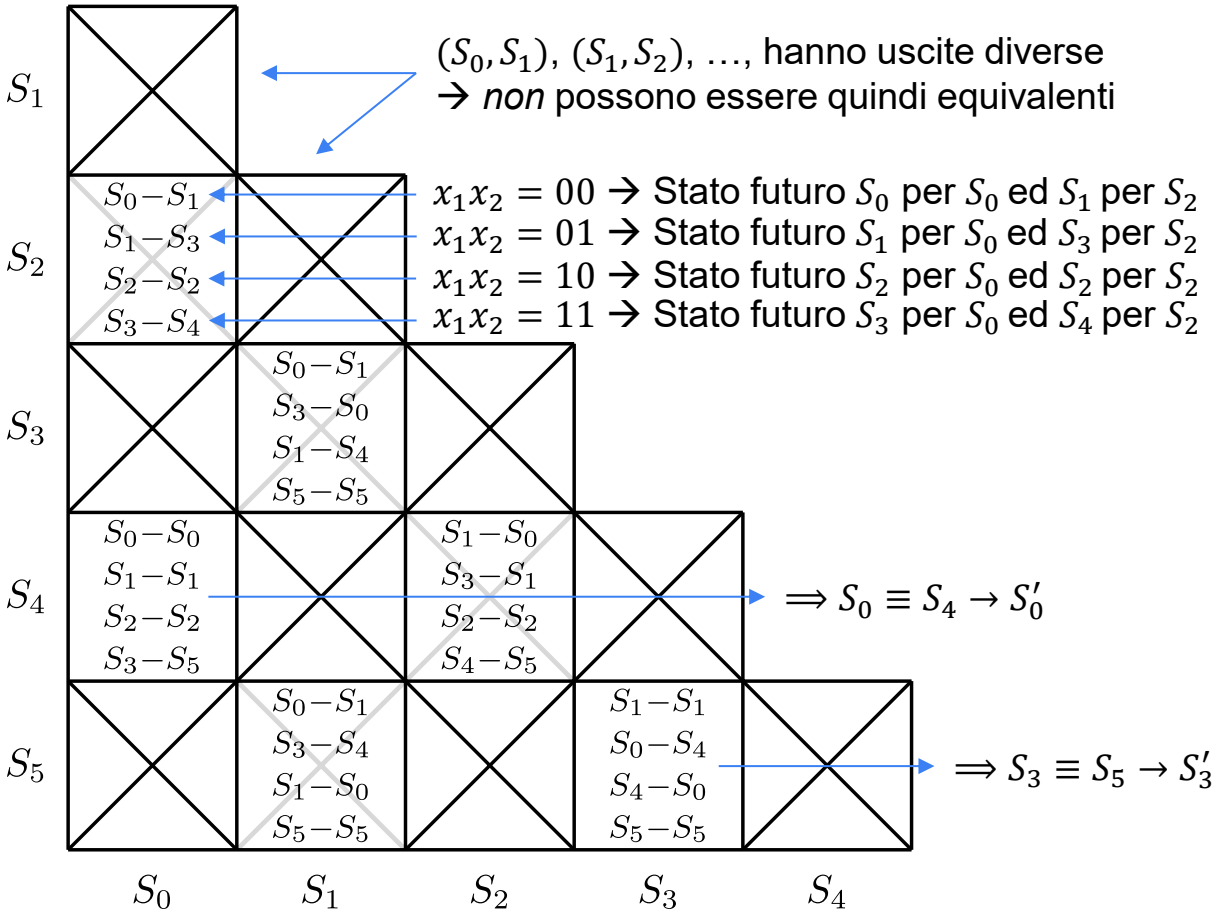


	Stato Futuro				
Stato Presente	00	01	10	11	Uscita
S_0	S_0	S_1	S_2	S_3	1
S_1	S_0	S_3	S_1	S_5	0
S_2	S_1	S_3	S_2	S_4	1
S_3	S_1	S_0	S_4	S_5	0
S_4	S_0	S_1	S_2	S_5	1
S_5	S_1	S_4	S_0	S_5	0

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della tabella delle implicazioni
Esempio con due variabili d'ingresso

- FSM con *due ingressi* e una *uscita*, rappresentata come macchina di Moore
  - Minimizzazione sulla base delle *implicazioni*



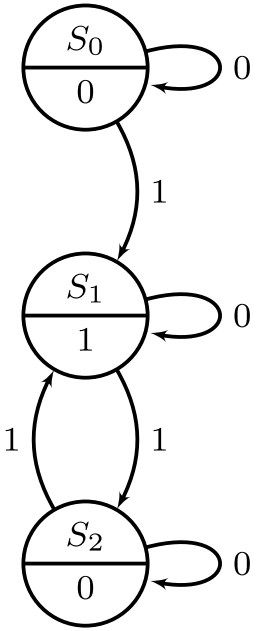
Stato Presente	Stato Futuro 00	01	10	11	Uscita
S_0	S_0	S_1	S_2	S_3	1
S_1	S_0	S_3	S_1	S_5	0
S_2	S_1	S_3	S_2	S_4	1
S_3	S_1	S_0	S_4	S_5	0
S_4	S_0	S_1	S_2	S_5	1
S_5	S_1	S_4	S_0	S_5	0

Stato Presente	Stato Futuro 00	01	10	11	Uscita
S_0'	S_0'	S_1	S_2	S_3'	1
S_1	S_0'	S_3'	S_1	S_3'	0
S_2	S_1	S_3'	S_2	S_0'	1
S_3'	S_1	S_0'	S_0'	S_3'	0

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Metodo della tabella delle implicazioni
Minimizza l'esempio del controllore di parità

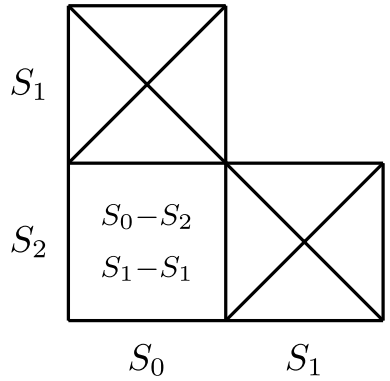
- Non ha i limiti del metodo della corrispondenza tra righe
- Esempio
  - L'equivalenza nel controllore di parità descritto in precedenza è *riconoscibile*
  - S_0 ed S_2 sono *equivalenti* poiché
    nessuna altra informazione contraddice tale conclusione



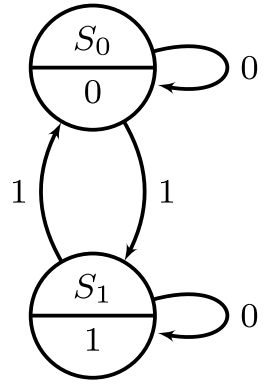


Stato Presente	Stato Futuro $x=0$	Stato Futuro $x=1$	Uscita
S_0	S_0	S_1	0
S_1	S_1	S_2	1
S_2	S_2	S_1	0

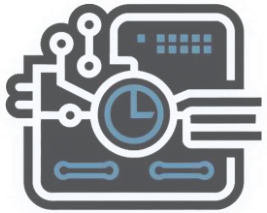








2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Ottimizzazione FSM
Codifica degli stati

- La *minimizzazione degli stati*
  → è il primo passo
- Scegliere la *codifica binaria* che *minimizza la logica combinatoria*
  → è il passo successivo

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Codifica degli stati

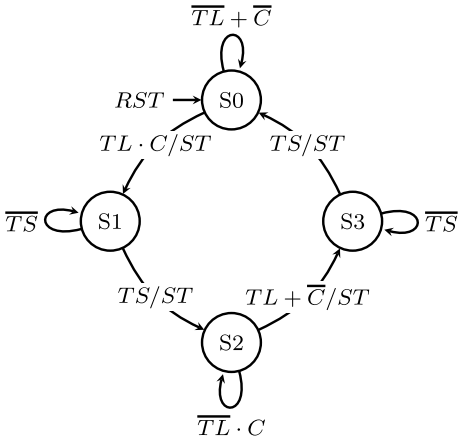
A1

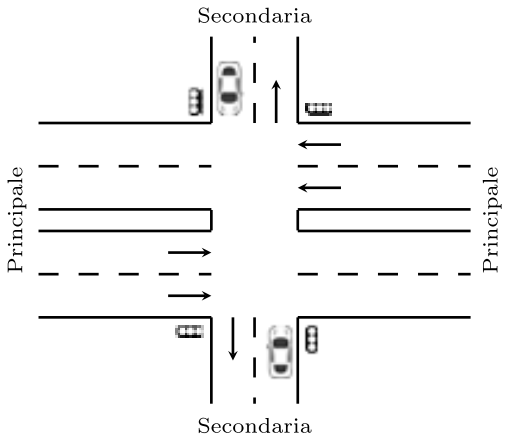
- Il *numero delle porte logiche* dipende (e anche molto) dalla *codifica degli stati*
- Esempio: una FSM con 4 stati comporta
  - 4 scelte per la codifica del primo stato
  - 3 scelte per la codifica del secondo stato
  - 2 scelte per il terzo
  - 1 *scelta obbligata* per lo stato rimanente

➔ Complessivamente: $4! = 24$ scelte possibili:

HG	HY	FG	FY		HG	HY	FG	FY
00	01	10	11		10	00	01	11
00	01	11	10		10	00	11	01
00	10	01	11		10	01	00	11
00	10	11	01		10	01	11	00
00	11	01	10		10	11	00	01
00	11	10	01		10	11	01	00
01	00	10	11		11	00	01	10
01	00	11	10		11	00	10	01
01	10	00	11		11	01	00	10
01	10	11	00		11	01	10	00
01	11	00	10		11	10	00	01
01	11	11	11		11	10	01	00

Possibili codifiche degli stati





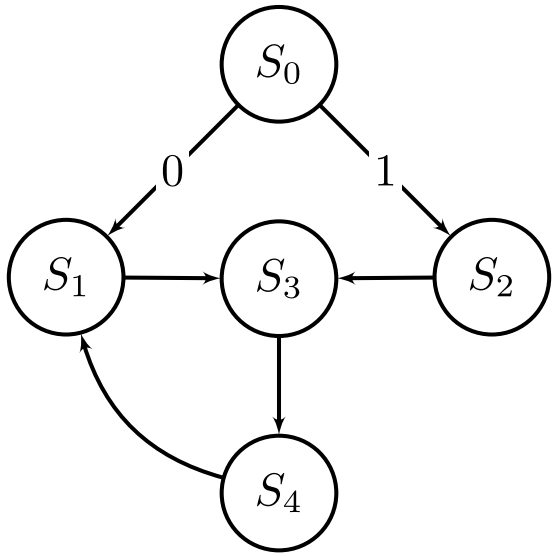
Ingressi			Stato Presente	Stato Futuro	Uscite		
C	TL	TS	Q_1Q_0	P_1P_0	ST	H_1H_0	F_1F_0
0	–	–	HG	HG	0	00	10
–	0	–	HG	HG	0	00	10
1	1	–	HG	HY	1	00	10
–	–	0	HY	HY	0	01	10
–	–	1	HY	FG	1	01	10
1	0	–	FG	FG	0	10	00
0	–	–	FG	FY	1	10	00
–	1	–	FG	FY	1	10	00
–	–	0	FY	FY	0	10	01
–	–	1	FY	HG	1	10	01

Codifica colori semaforo

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

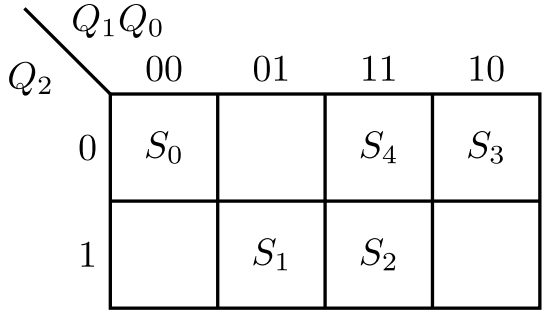
Codifica degli stati
Metodi euristici

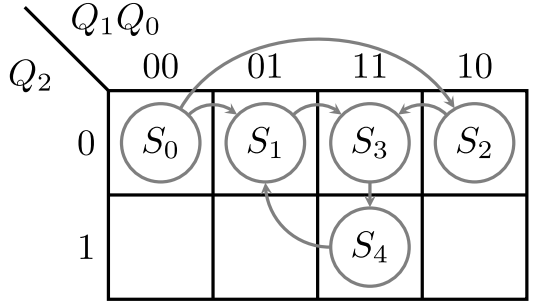
- Obiettivo: assegnare *codici vicini* a stati con *relazioni di stato presente e futuro*
- Metodi euristici manuali, stile «carta e penna»
  - Usano mappe simili a quelle di Karnaugh (solo 1 bit differenza tra codifiche in celle adiacenti)



	Codifica 1		
Stato	Q_2	Q_1	Q_0
S_0	0	0	0
S_1	1	0	1
S_2	1	1	1
S_3	0	1	0
S_4	0	1	1

	Codifica 2		
Stato	Q_2	Q_1	Q_0
S_0	0	0	0
S_1	0	0	1
S_2	0	1	0
S_3	0	1	1
S_4	1	1	1



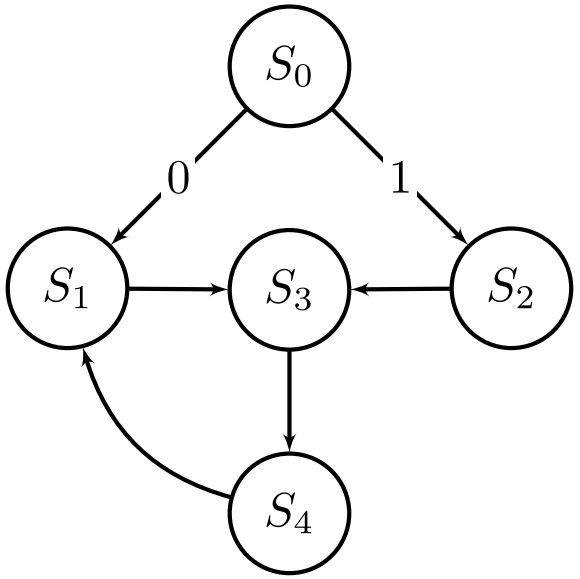


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Codifica degli stati
Metriche di confronto: distanza di Hamming

- *Distanza di Hamming* è una metrica di *distanza tra codifiche* degli stati
  - Misura il *numero di bit che variano* tra le rappresentazioni di due stati *coinvolti in una transizione* nella FSM

➔ *Minimizzare la distanza di Hamming complessiva* per il sistema



	Codifica 1		
Stato	Q_2	Q_1	Q_0
S_0	0	0	0
S_1	1	0	1
S_2	1	1	1
S_3	0	1	0
S_4	0	1	1

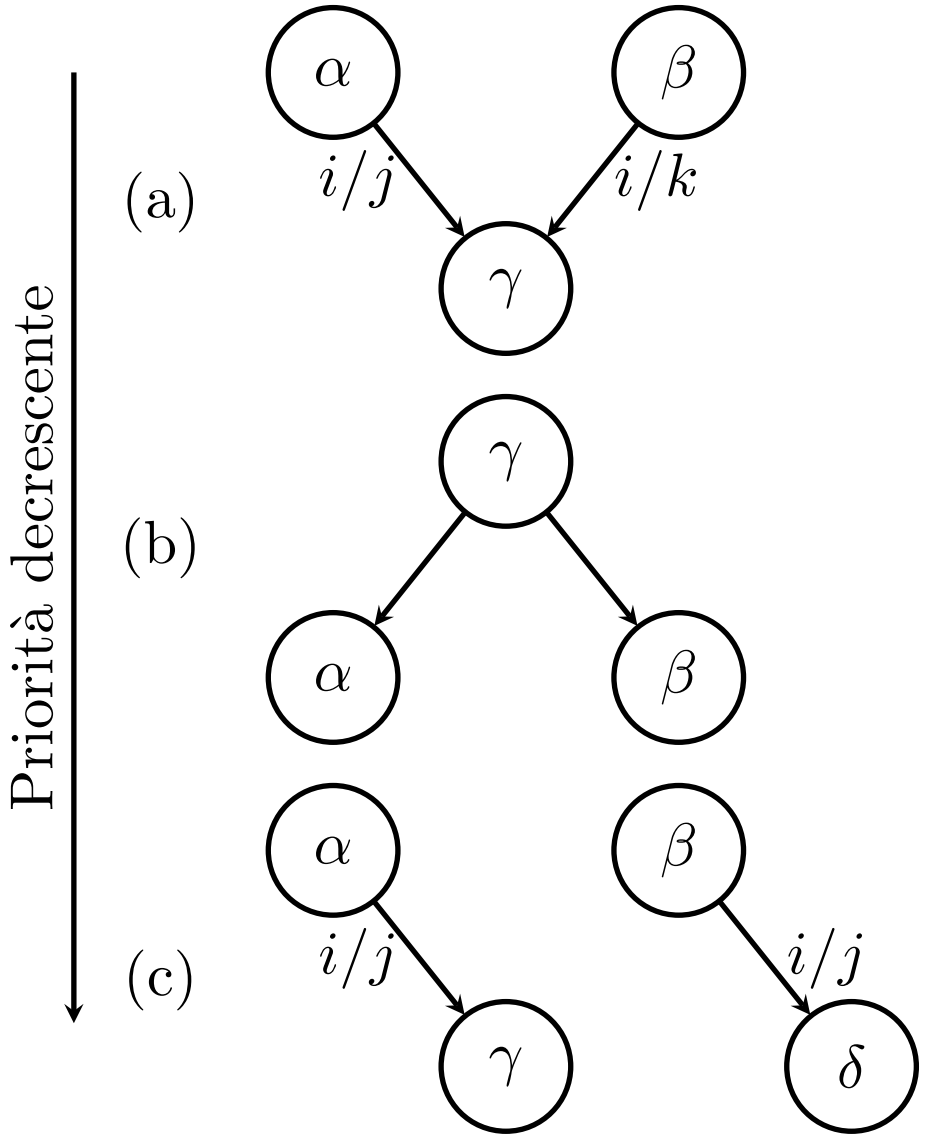
	Codifica 2		
Stato	Q_2	Q_1	Q_0
S_0	0	0	0
S_1	0	0	1
S_2	0	1	0
S_3	0	1	1
S_4	1	1	1

	Δ_1	Δ_2
$S_0 \rightarrow S_1$	2	1
$S_0 \rightarrow S_2$	3	1
$S_1 \rightarrow S_3$	3	1
$S_2 \rightarrow S_3$	2	1
$S_3 \rightarrow S_4$	1	1
$S_4 \rightarrow S_1$	2	2
Σ	13	7

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Codifica degli stati
Metodi euristici

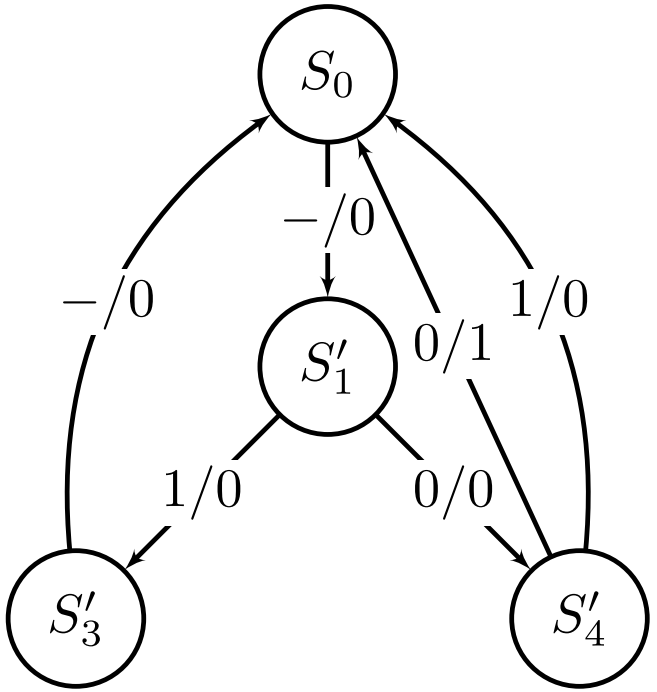
- *Euristiche alternative*
  - Tengono conto anche del comportamento di ingresso/uscita, oltre che dello stato
- Si assegnano *codici adiacenti* privilegiando certe condizioni, con priorità differenti
  1. Stati con lo *stesso stato futuro* (a)
     - Raggruppa gli 1 nella mappa dello stato futuro
  2. Stati con lo *stesso stato precedente* (b)
     - Può favorire il raggruppamento di 1 nella mappa dello stato futuro
  3. Stati con la *stessa uscita* (c)
     - Raggruppa gli 1 nella mappa delle uscite



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Codifica degli stati
Metodi euristici

- FSM di riferimento: identificatore di sequenza a 3 bit
- Priorità *massima*:
  stesso *stato futuro*
  - $(S'_3 - S'_4)$
- Priorità *media*:
  stesso *stato precedente*
  - $(S'_3 - S'_4)$
- Priorità *minima*:
  stesse *uscite*
  - $0/0$: (S_0, S'_1, S'_3)
  - $1/0$: (S_0, S'_1, S'_3, S'_4)



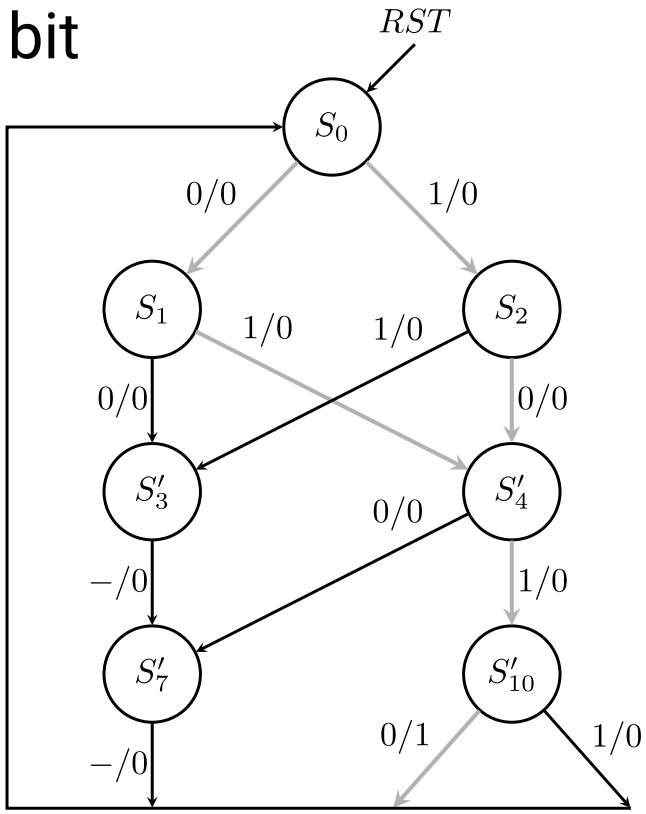
Q_1 \ Q_0	0	1
0	S_0	S'_1
1	S'_3	S'_4

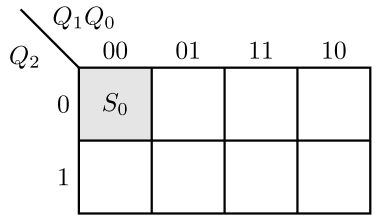
Q_1 \ Q_0	0	1
0	S_0	S'_3
1	S'_1	S'_4

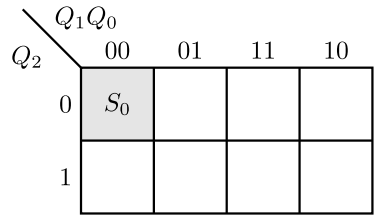
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

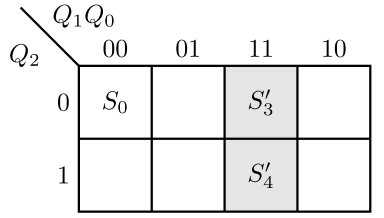
Codifica degli stati
Metodi euristici

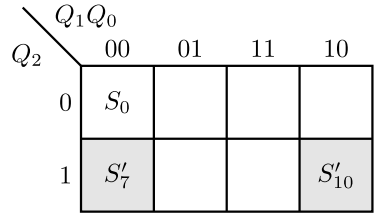
- FSM di riferimento:
  identificatore di sequenza a 4 bit
- Priorità *massima*:
  stesso *stato futuro*
  - $0/0$: $(S'_3 - S'_4)$, $1/0$: $(S'_7 - S'_{10})$
- Priorità *media*:
  stesso *stato precedente*
  - $(S_1 - S_2)$, $(S'_3 - S'_4)$, $(S'_7 - S'_{10})$
- Priorità *minima*: stesse *uscite*
  - $0/0$: $(S_0, S_1, S_2, S'_3, S'_4, S'_7)$
  - $1/0$: $(S_0, S_1, S_2, S'_3, S'_4, S'_7)$
- Obiettivi
  - Impostare S_0 come stato iniziale 000
  - Tenere *vicine* le coppie a *priorità massime* e poi *medie*

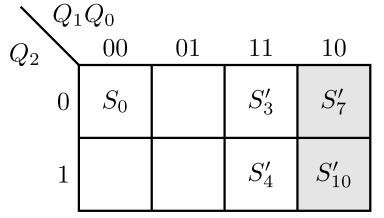


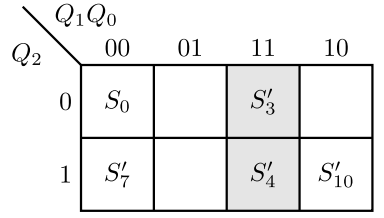


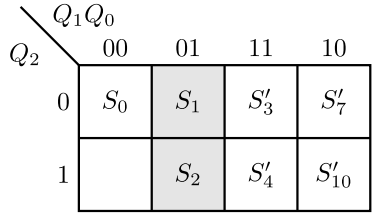




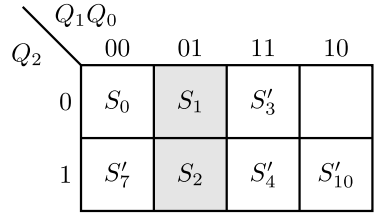








(a)



(b)

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

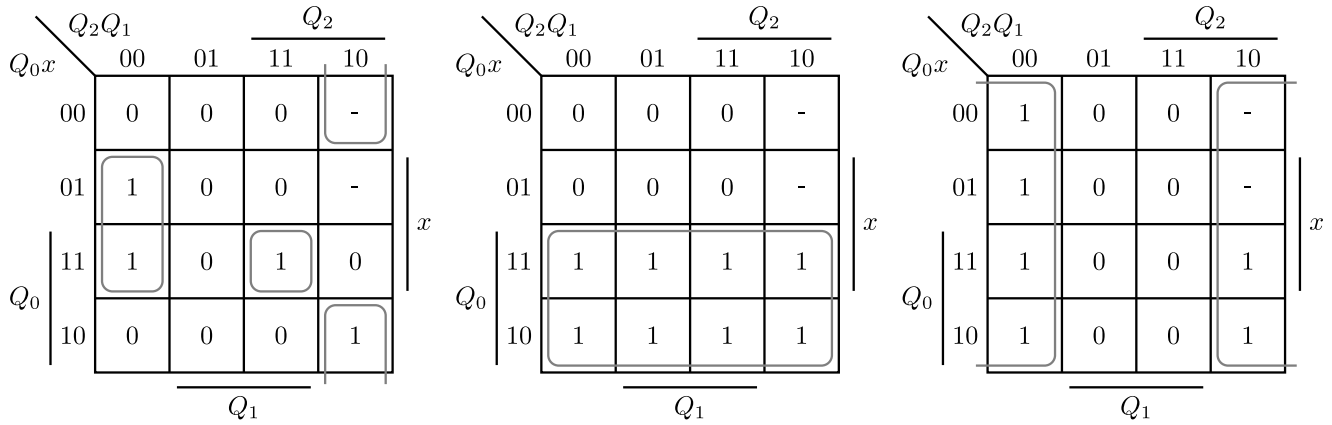
Codifica degli stati
Metodi euristici

- Le *adiacenze* hanno un impatto diretto sulla mappa degli stati
  - La codifica (a) *raggruppa meglio gli* 1 nella mappa dello stato futuro

(a)

Stato Presente		Stato Futuro $x=0$	Stato Futuro $x=1$
(S_0)	000	001	101
(S_1)	001	011	111
(S_2)	101	111	011
(S'_3)	011	010	010
(S'_4)	111	010	110
(S'_7)	010	000	000
(S'_{10})	110	000	000

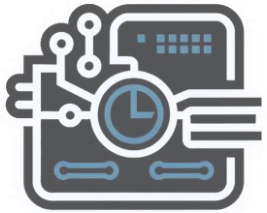
Stato Presente		Stato Futuro $x=0$	Stato Futuro $x=1$
(S_0)	000	001	010
(S_1)	001	011	100
(S_2)	010	100	011
(S'_3)	011	101	101
(S'_4)	100	101	110
(S'_7)	101	000	000
(S'_{10})	110	000	000



$Q_i \rightarrow$ stato presente, $P_i \rightarrow$ stato futuro

P_2 P_1 P_0

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Codifica degli stati
One-hot

- Codifica «one-hot» rappresenta n *stati* usando n *flip-flop*
- Esempio
  - Per 4 stati, le codifiche possibili sono 0001, 0010, 0100 e 1000
- In ogni stato, *una sola* variabile di stato vale 1
- Le *configurazioni non one-hot* sono codici non usati e possono essere trattate come *don't care* nella minimizzazione
- Le *equazioni di stato futuro* risultano spesso più semplici da derivare
- La *logica combinatoria* può semplificarsi, ma il *numero di flip-flop aumenta*

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

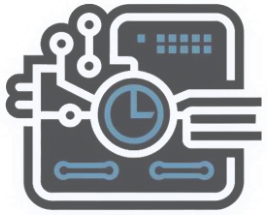
Codifica degli stati
One-hot

- Esempio
  - Quattro stati, A, B, C e D, con codifiche, rispettivamente, 0001, 0010, 0100 e 1000
  - Assumendo una realizzazione con flip-flop di tipo D, le equazioni di stato futuro si possono «leggere» direttamente dai valori a 1 nella relativa colonna della tabella degli stati

		Stato Futuro		Uscita	
Stato Presente		$x=0$	$x=1$	$x=0$	$x=1$
(A)	0001	0001	0010	0	0
(B)	0010	0001	0100	0	0
(C)	0100	1000	0100	0	0
(D)	1000	0001	0010	0	1

$$D_0 = \overline{x}(Q_0 + Q_1 + Q_3) = \overline{x}\,\overline{Q_2}$$
$$D_1 = x(Q_0 + Q_3)$$
$$D_2 = x(Q_1 + Q_2) \qquad = x\left(\overline{Q_0 + Q_3}\right)$$
$$D_3 = \overline{x}Q_2$$
$$U = xQ_3$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

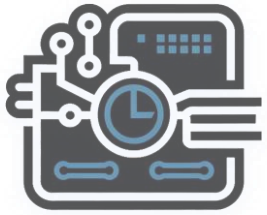


Scelta dei flip-flop
Considerazioni e procedura

A2

- Le implementazioni possono cambiare in base al tipo di flip-flop: numero di porte, connessioni, semplicità realizzativa ecc.
- Flip-flop di tipo D
  - Realizzazione più semplice, meno connessioni
- Flip-flop di tipo J-K
  - Meno porte, ma più connessioni
    → Argomento di approfondimento individuale
- Procedura di technology mapping
  - Data una codifica degli stati, derivare le *mappe dello stato futuro* dalla tabella di transizione codificata
  - Usare le tabelle di eccitazione dei flip-flop per ottenere le loro *funzioni di eccitazione*
  - *Minimizzare* le funzioni di eccitazione

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Scelta dei flip-flop
Esempio: tabelle di transizione codificate

- FSM di riferimento: identificatore di sequenza a 4 bit

Stato Presente		Stato Futuro $x=0$	Stato Futuro $x=1$	Uscita $x=0$	Uscita $x=1$
(S_0)	000	S_1	S_2	0	0
(S_1)	011	S'_3	S'_4	0	0
(S_2)	010	S'_4	S'_3	0	0
(S'_3)	101	S'_7	S'_7	0	0
(S'_4)	111	S'_7	S'_{10}	0	0
(S'_7)	100	S_0	S_0	0	0
(S'_{10})	110	S_0	S_0	1	0

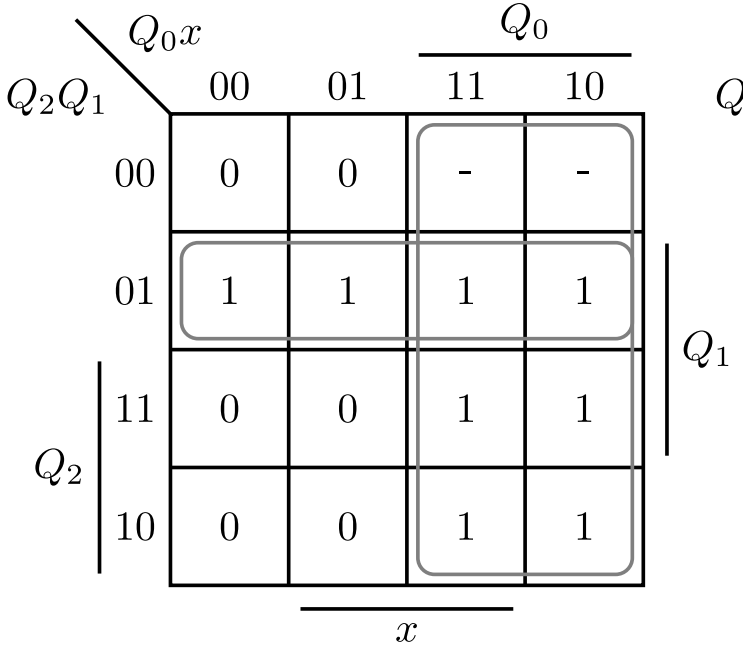


Stato Presente	Stato Futuro $x=0$	Stato Futuro $x=1$
000	011	010
001	−−−	−−−
010	111	101
011	101	111
100	000	000
101	100	100
110	000	000
111	100	110

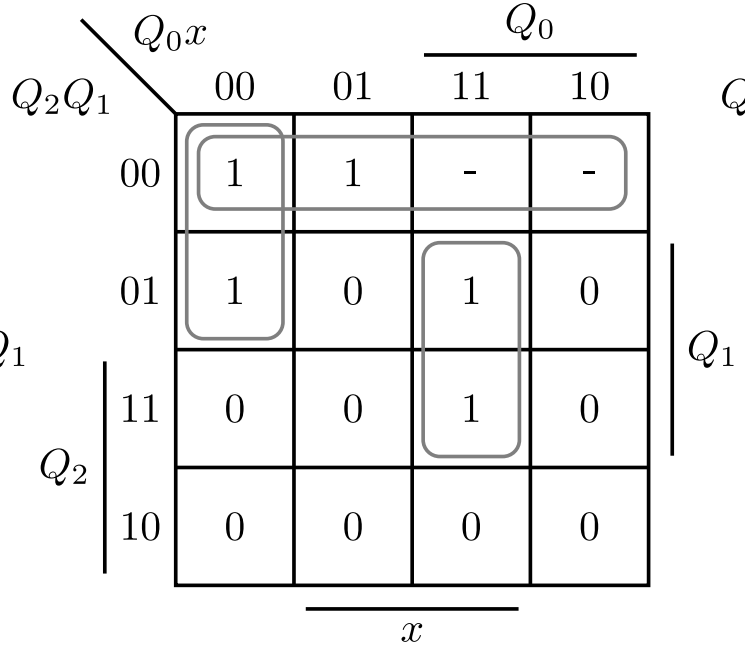
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Scelta dei flip-flop
Esempio: funzioni di eccitazione minimizzate

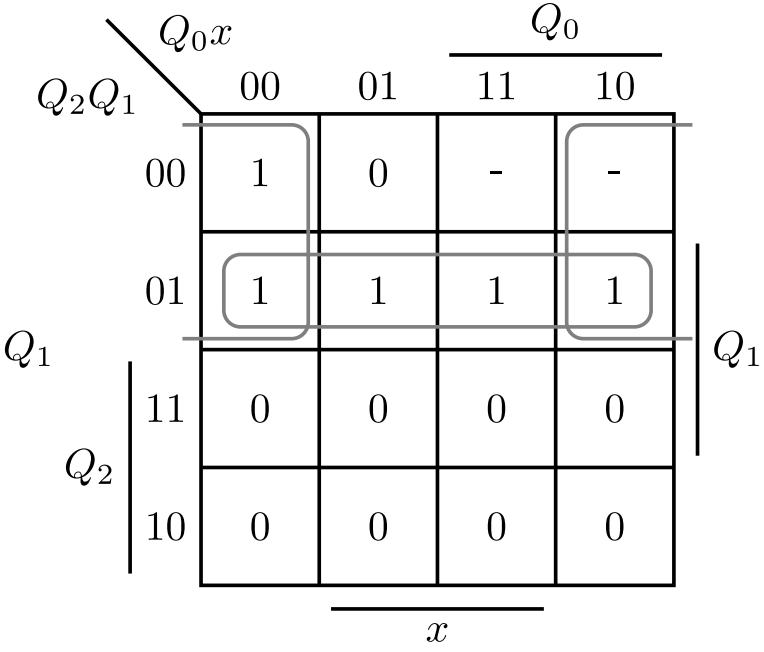
- Realizzazione con flip-flop di tipo D
  - ➔ 6 termini prodotto ($\overline{Q_2}Q_1$ si ripete) e 15 letterali



$$D_{Q_2} = \overline{Q_2}Q_1 + Q_0$$



$$D_{Q_1} = \bar{Q}_2\bar{Q}_1 + Q_1Q_0x + \bar{Q}_2\bar{Q}_0\bar{x}$$



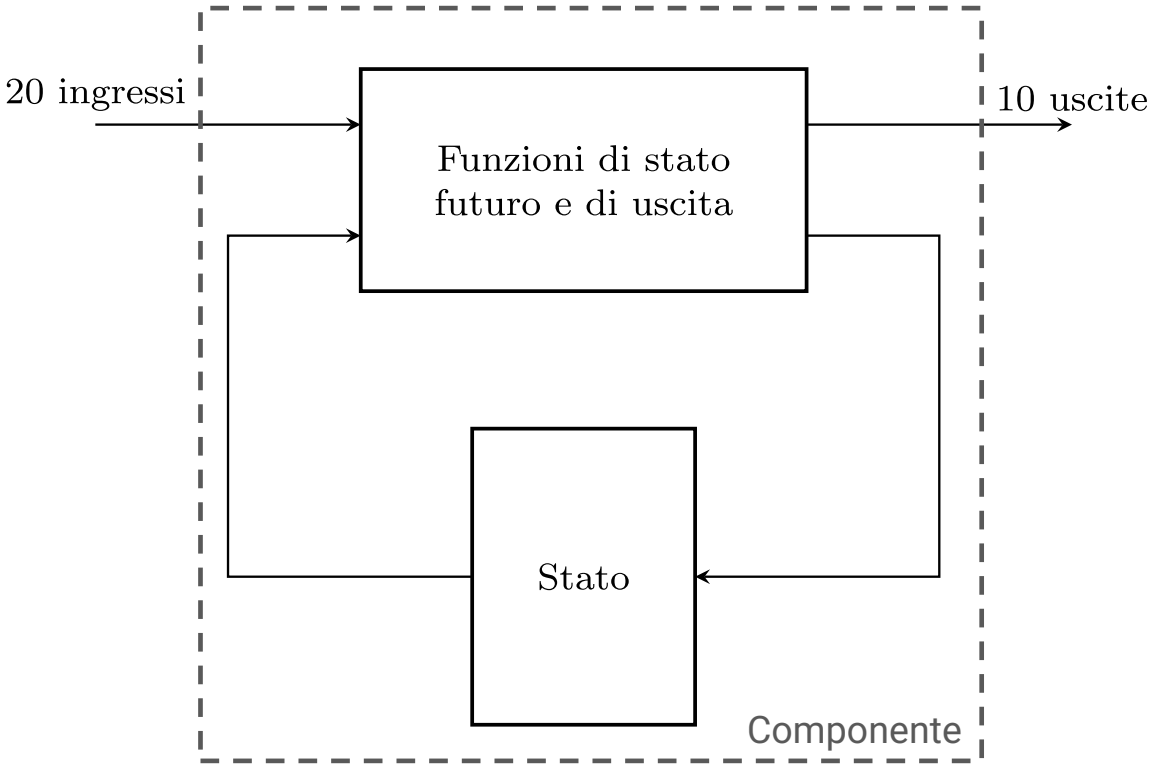
$$D_{Q_0} = \overline{Q_2}Q_1 + \overline{Q_2}\overline{x}$$

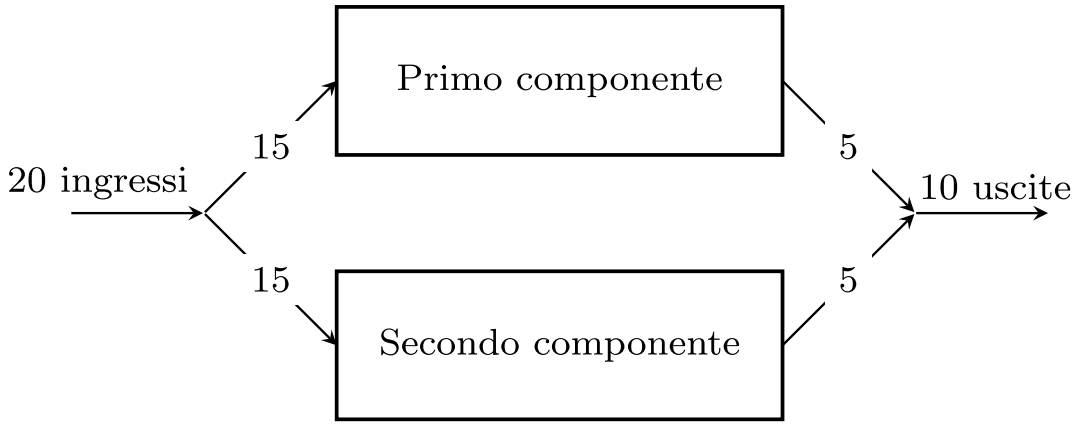
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Decomposizione di FSM

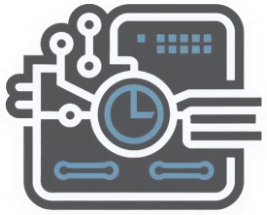
A3

- In alcuni casi può essere conveniente *suddividere in parti* una FSM
  - Ad esempio, nel caso della realizzazione su *componenti programmabili*
    - Numero limitato di *pin di ingresso/uscita*
    - Numero limitato di *termini prodotto*, *flip-flop* e *altre risorse* programmabili





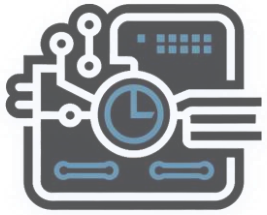
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Rappresentazioni alternative per FSM

- In molte situazioni, i diagrammi degli stati non sono sufficienti
  - Non sono abbastanza *flessibili* per *grandi FSM*
  - Non sono *adatti* per *sviluppo graduale* della FSM
  - Non descrivono in *modo ovvio* un algoritmo, cioè una ben specifica *sequenza di operazioni* basate sui dati di ingresso
  - Manca la *separazione tra controllo e dati*
- Spostamento graduale verso *rappresentazioni simili ad un programma*
  - Rappresentazione ad algorithmic state machine (ASM)
  - Linguaggi di descrizione dell'hardware (hardware description languages, HDL)

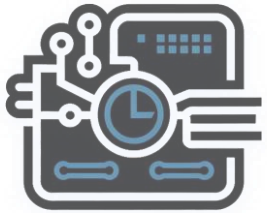
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Algorithmic state machine (ASM)

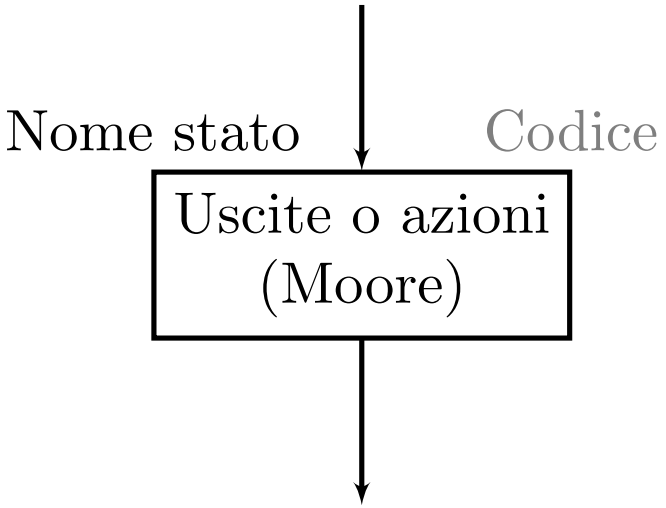
- ASM è una rappresentazione simile a un diagramma di flusso *adatta a modellare FSM*
- Si possono usare *tre tipi di componenti*
  - *Stato*
  - *Decisione*
  - *Uscita*
- La FSM è *esattamente* in un *solo stato alla volta*
- C'è un *singolo ingresso* in uno stato
- C'è un *singolo cammino di uscita* da uno stato, per *ogni combinazione degli ingressi*
- Le *uscite* possono essere attive, basse o alte, con *risposta immediata* o *ritardata* nel tempo
- L'ASM può modellare sia macchine di Mealy, sia di Moore, sia ibride

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

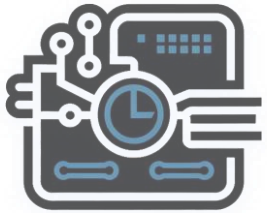


Algorithmic state machine (ASM)
Blocco di stato

- Rappresenta uno *stato del sistema*
  - Caratterizzato dal relativo *nome*, in alto a sinistra
  - Opzionalmente caratterizzato dalla *codifica* dello stato, in alto a destra
- Equivalente a un *nodo in un diagramma a stati* o a una *riga della tabella degli stati*
- Contiene *azioni* che interessano i *registri* o *segnali di output*
- Gli *output del tipo Moore* sono riportati all'interno dei blocchi di stato
- Sono solitamente riportati *solo i segnali asseriti* in un certo stato, non l'elenco completo

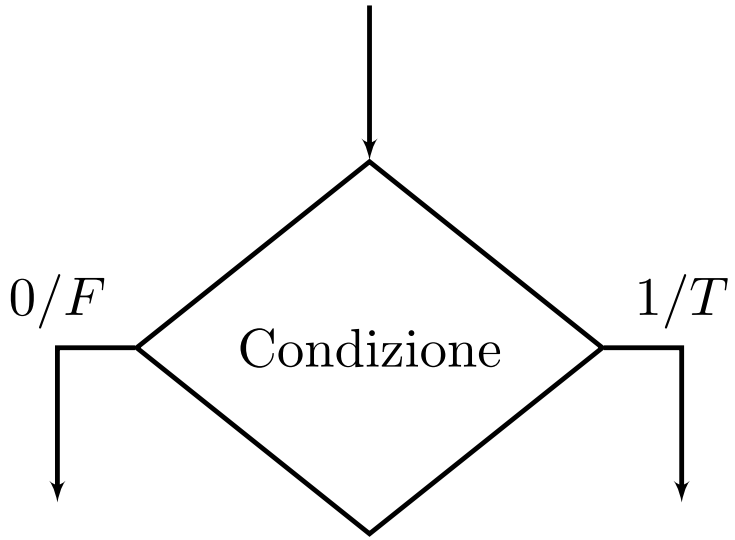


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

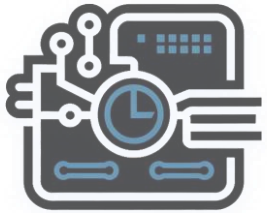


Algorithmic state machine (ASM)
Blocco di decisione

- Rappresenta una *condizione da valutare* lungo un certo cammino nel flusso della ASM
- Sulla base del *risultato della valutazione*, il flusso procede lungo il cammino associato a quello specifico ramo
- La condizione da valutare può coinvolgere *uno o più input* del sistema

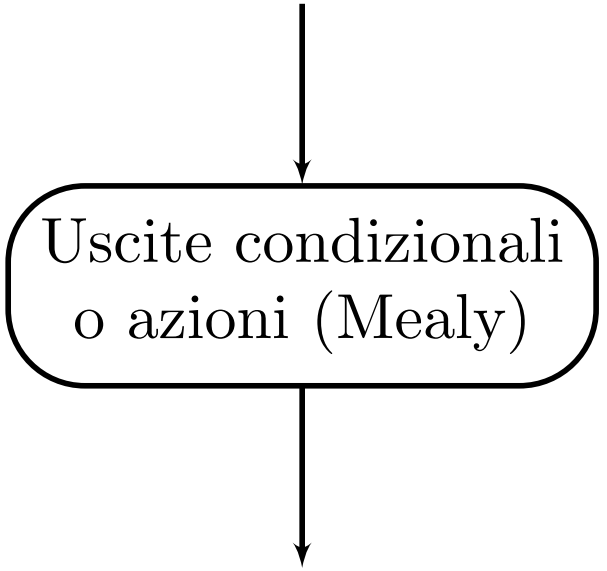


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Algorithmic state machine (ASM)
Blocco di output condizionale

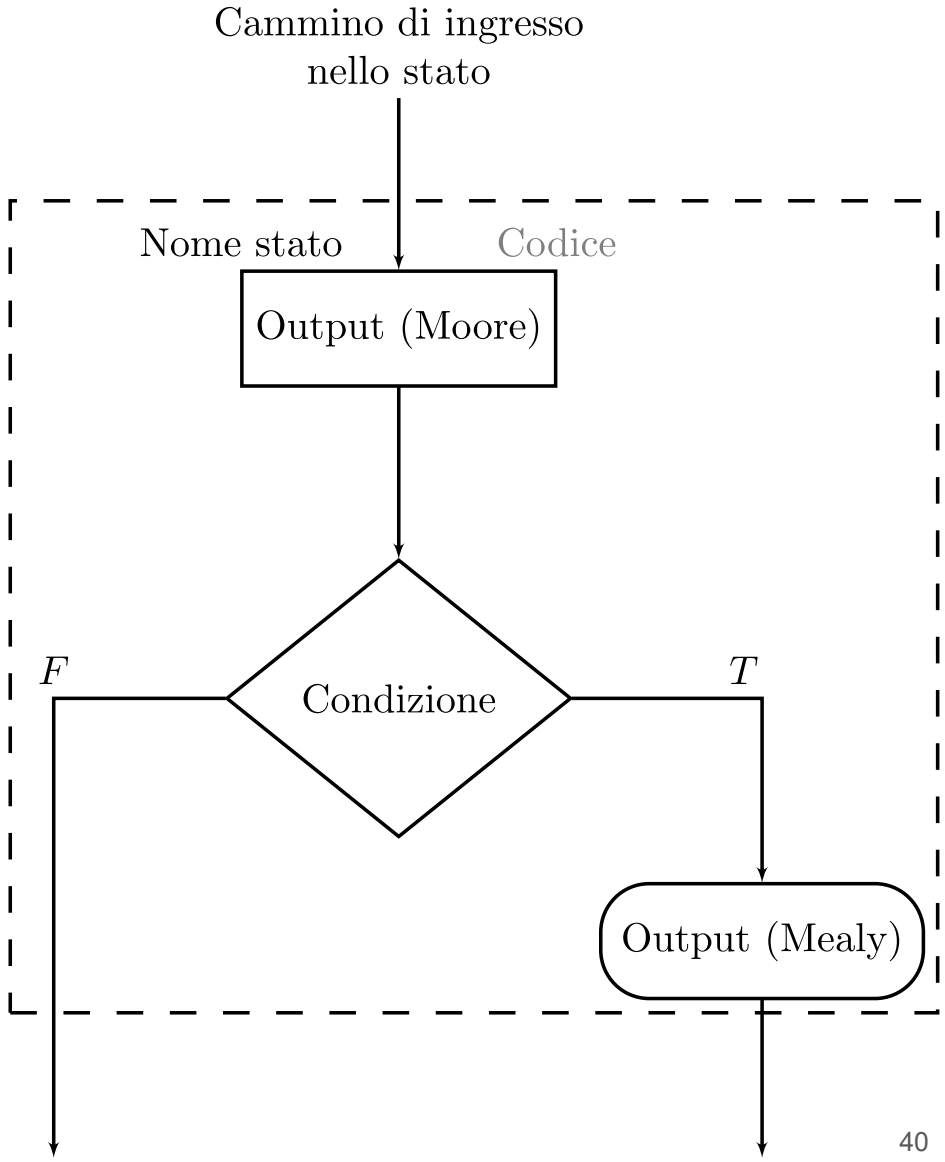
- Rappresenta un blocco di *output condizionale*
- Gli *output del tipo Mealy* sono riportati in questa categoria di blocchi
- *La condizione* che specifica se un certo output sia generato o meno è riportata nel *blocco di decisione che li precede*



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Algorithmic state machine (ASM)
Blocco base e regole generali

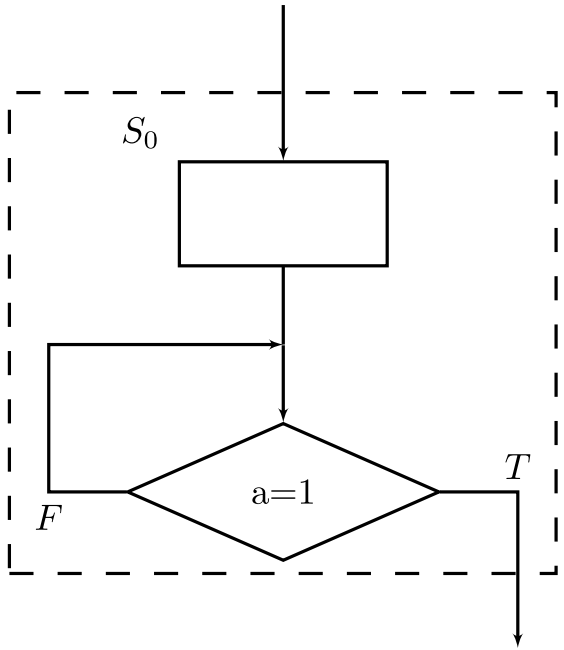
- Un *singolo blocco di stato*
- Uno o più *blocchi di decisione* (in cascata, opzionali) con cammini etichettati
- Uno o più *blocchi di output condizionale* (opzionali)
- Le transizioni
  - Sono *governate dal clock*
  - Avvengono tra *blocchi base*
- Deterministico: da ogni stato ASM, per *ogni combinazione degli ingressi*, è definito un *unico cammino di evoluzione*
- *Ogni ciclo* nel diagramma ASM deve includere *un blocco di stato*

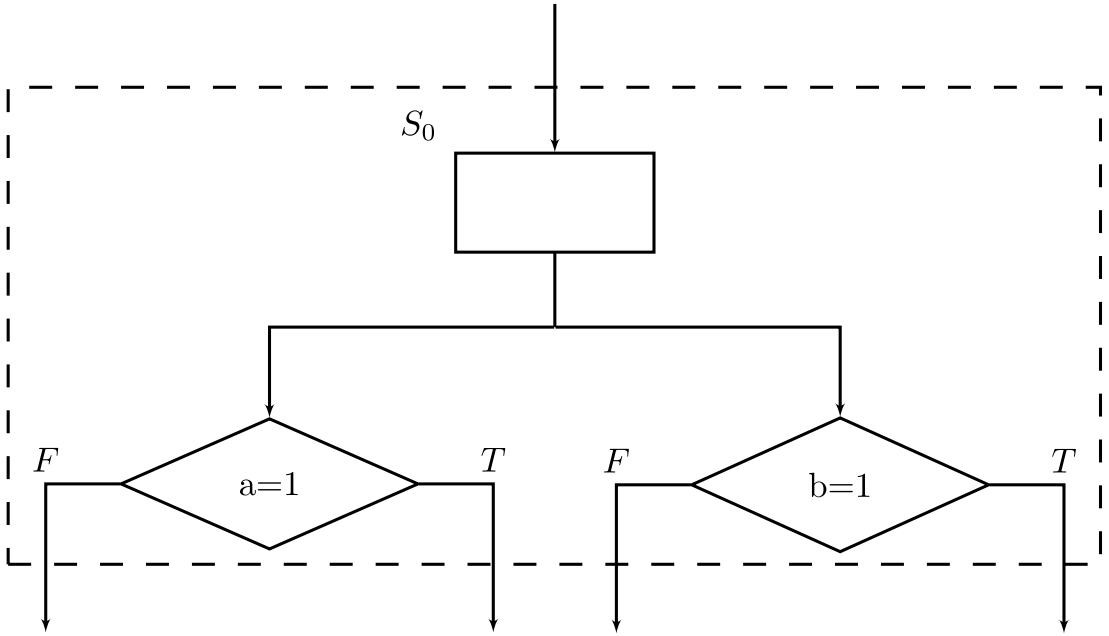


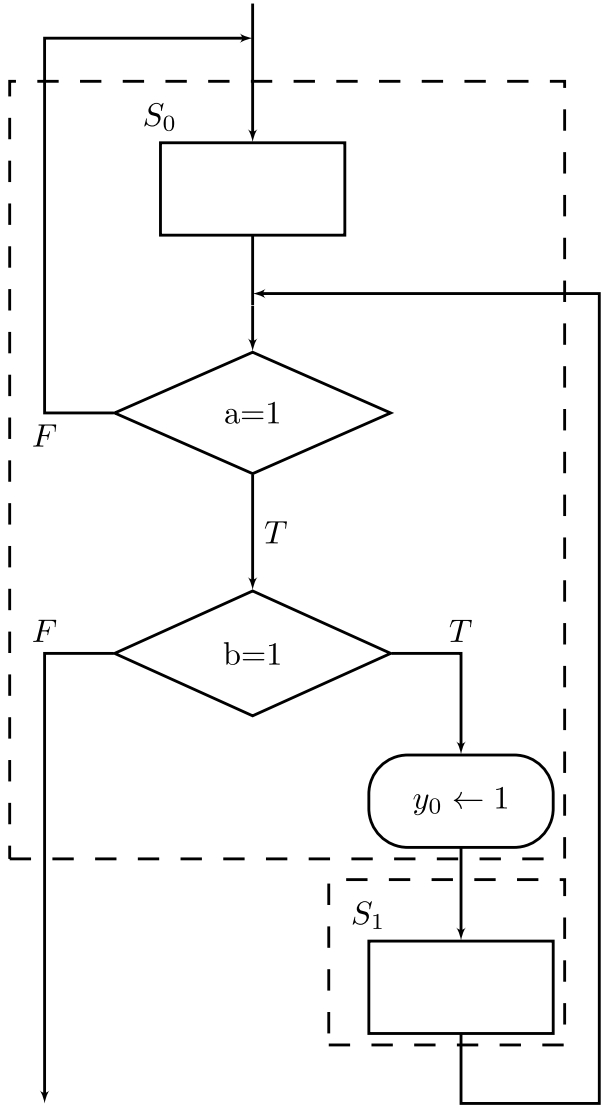
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Algorithmic state machine (ASM)
Esempi

- I due esempi a sinistra e al centro *non* rappresentano configurazioni valide
- L'unica struttura *corretta* tra quelle proposte è lo schema più a *destra*



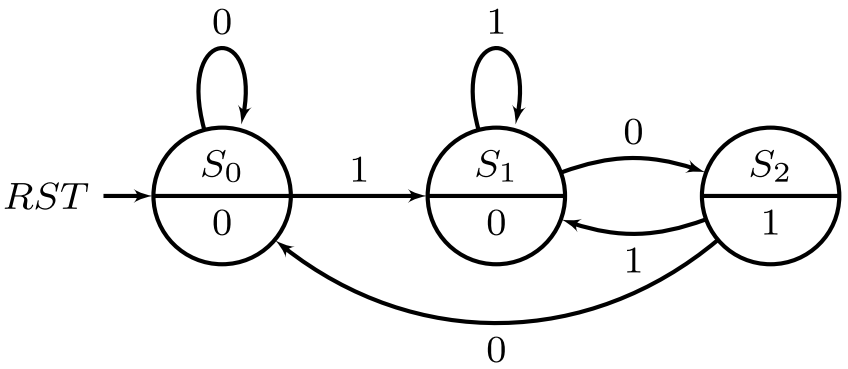


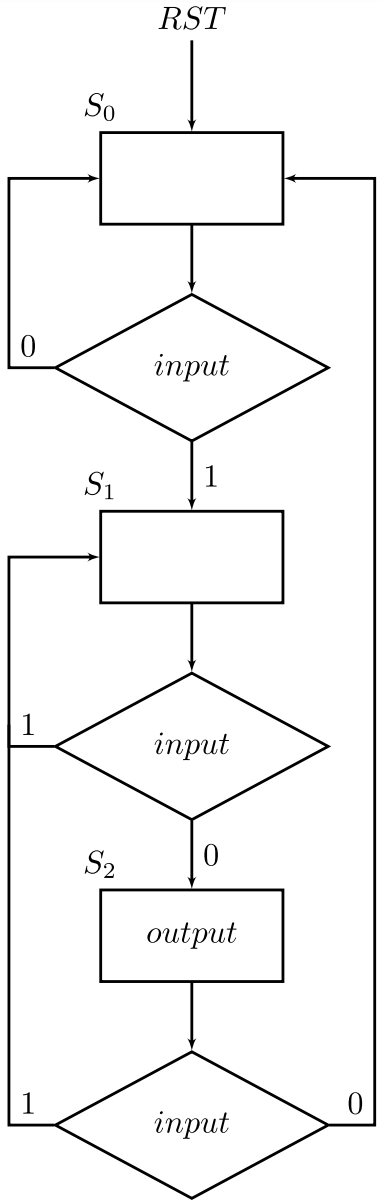


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Algorithmic state machine (ASM)
Macchina di Moore

- FSM di riferimento: riconoscitore di sequenza 10
  - S_0: nessun elemento della sequenza osservato
  - S_1: sequenza 1 riconosciuta
  - S_2: sequenza 10 riconosciuta

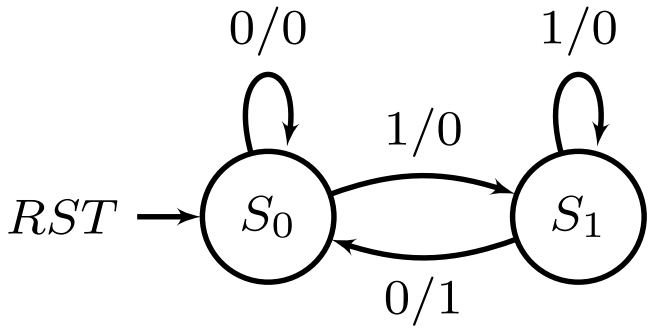


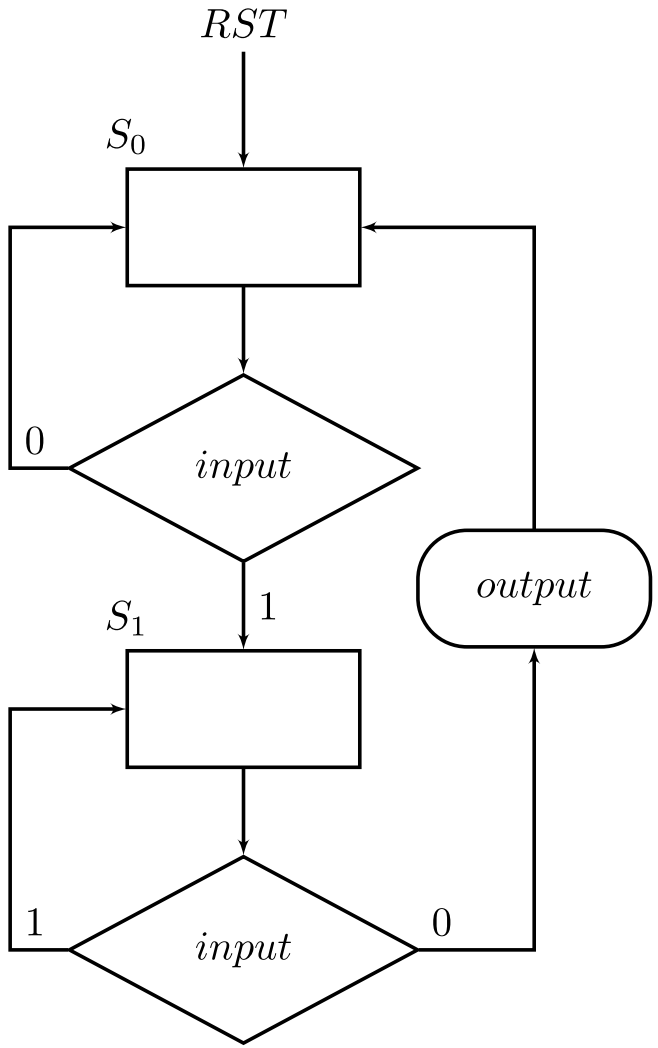


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Algorithmic state machine (ASM)
Macchina di Mealy

- FSM di riferimento:
  - Riconoscitore di sequenza 10
  - S_0: nessun elemento della sequenza osservato
  - S_1: sequenza 1 riconosciuta



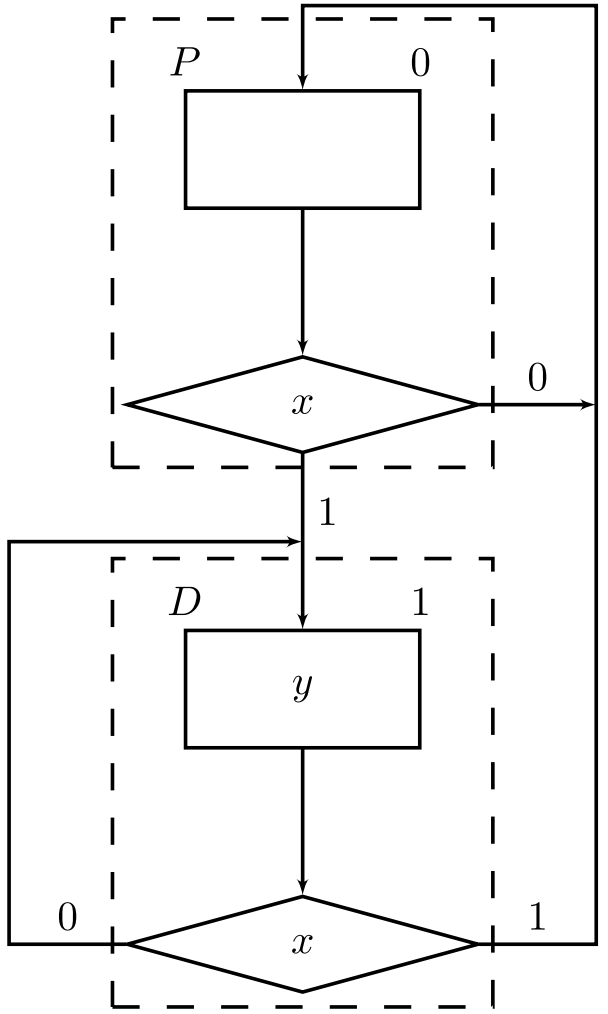


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio ASM
Controllore di parità

- L'FSM di riferimento ha ingresso x e uscita y
- *Lista delle uscite vuota* in un certo stato (es, P) implica y *non attivo*
- L'uscita y è invece asserita nello stato «dispari» (D)
- La tabella degli stati/transizioni è ricavabile seguendo tutti i cammini all'interno dell'ASM di riferimento

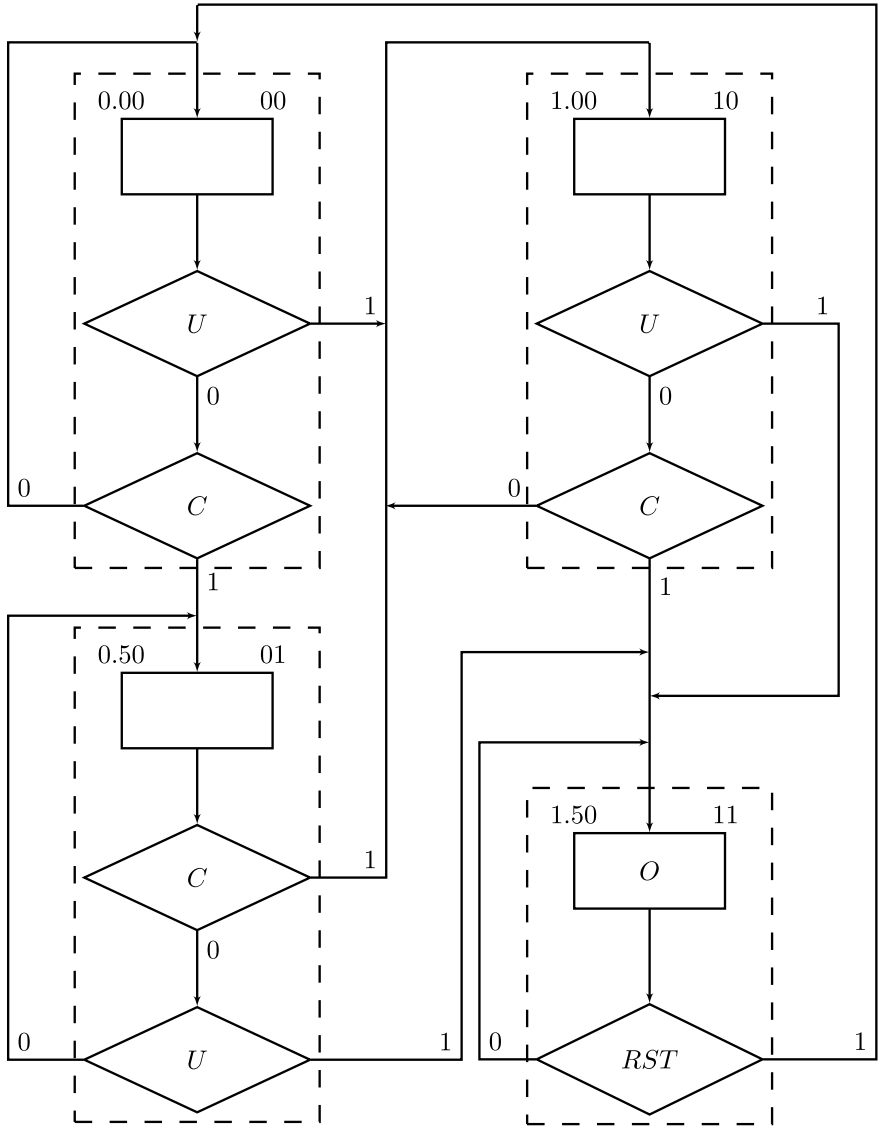
Ingresso	Stato Presente	Stato Futuro	Uscita
0	0	0	0
1	0	1	0
0	1	1	1
1	1	0	1



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio ASM
Distributore automatico

- L'FSM di riferimento ha
  - Due ingressi che rilevano le monete da 1 € (U) e $0{,}50$ € (C)
  - Il segnale di reset (RST)
  - Un'uscita legata all'erogazione del prodotto (O)
- L'ASM ha quattro stati attraversati sequenzialmente in base alle monete inserite
- L'uscita è asserita solo al raggiungimento dello stato che significa totale sufficiente per erogare il prodotto ($1{,}50$ € o più)
- Per ogni stato sono riportate
  - La codifica (in alto a destra)
  - Il nome (in alto a sinistra)



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio ASM
Controllore semaforico

- Caso più complesso

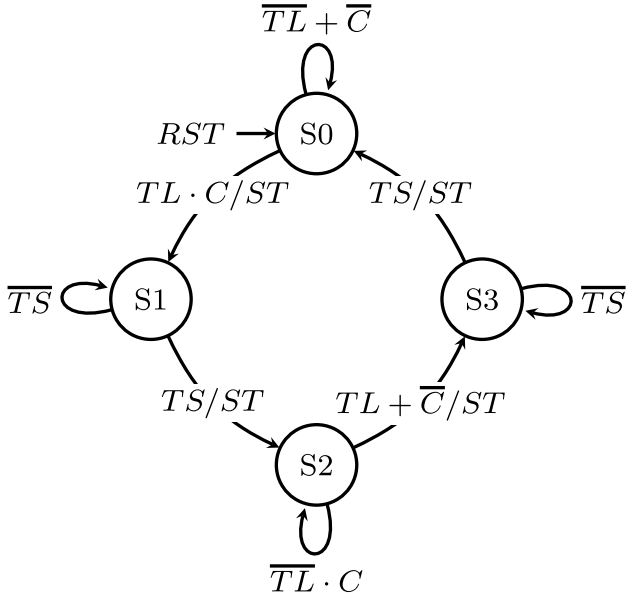
→ Costruzione dell'ASM per passi

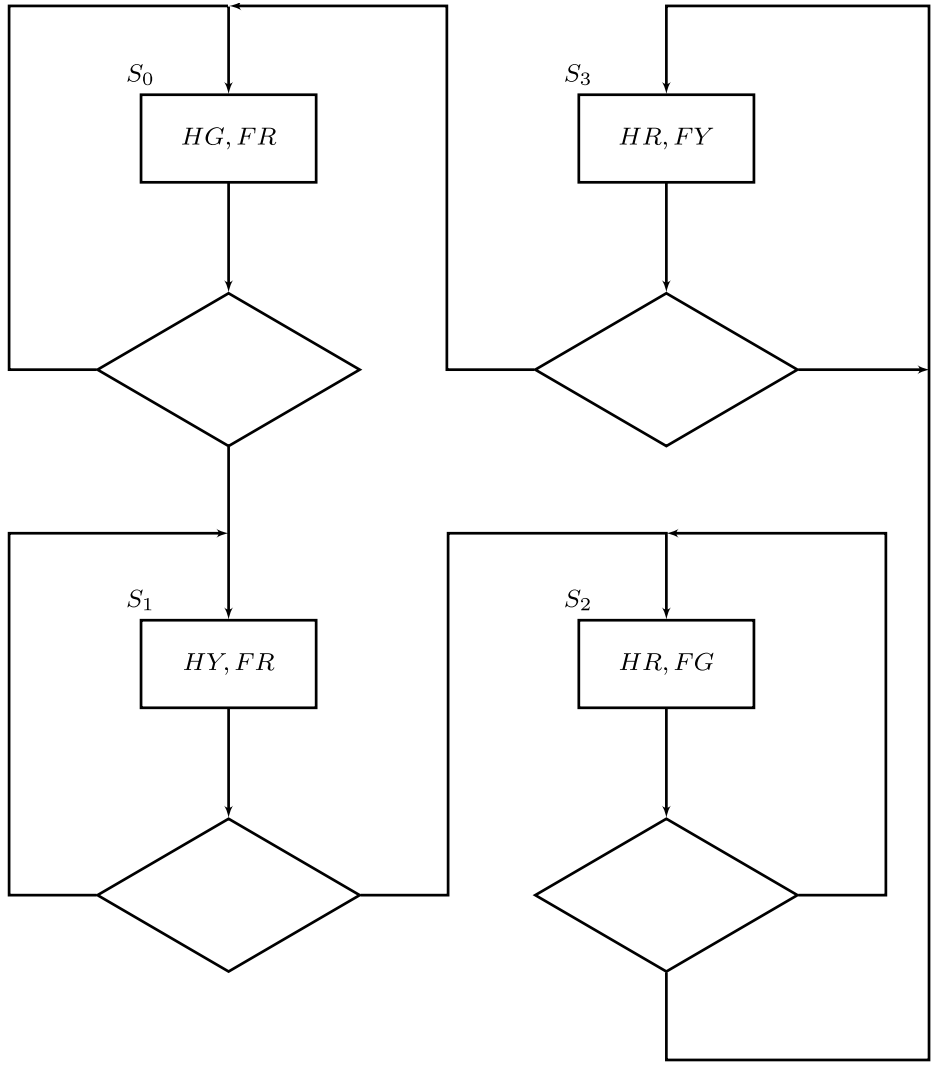
- Descrizione iniziale *molto basilare degli stati*, ignorando le *condizioni di transizione*
- Passo iniziale
  - Sequenza di stati
  - Uscite di base

Segnale di ingresso	Descrizione
RST	Porta la FSM allo stato iniziale
C	Presenza di veicolo su strada secondaria
TS	Fine intervallo breve
TL	Fine intervallo lungo

Segnale di uscita	Descrizione
HG, HY, HR	Attiva luce verde/gialla/rossa principale
FG, FY, FR	Attiva luce verde/gialla/rossa secondaria
ST	Inizio intervallo di tempo

Stato	Descrizione
S_0	Verde principale (Rosso secondaria)
S_1	Giallo principale (Rosso secondaria)
S_2	Verde secondaria (Rosso principale)
S_3	Giallo secondaria (Rosso principale)





2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

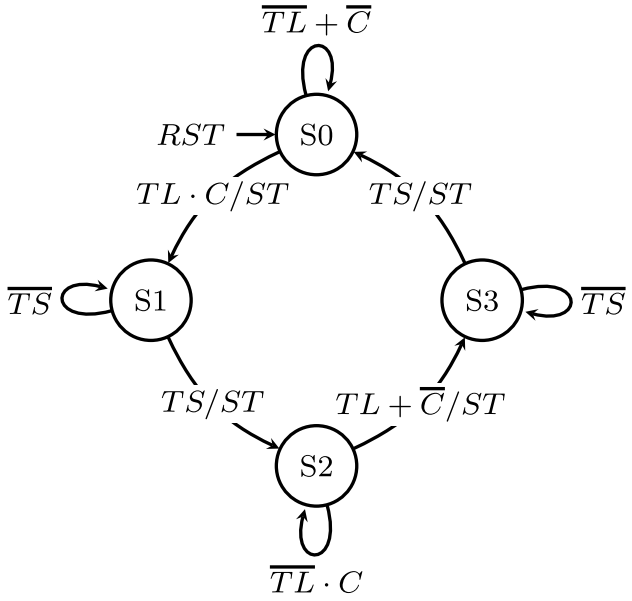
Esempio ASM
Controllore semaforico

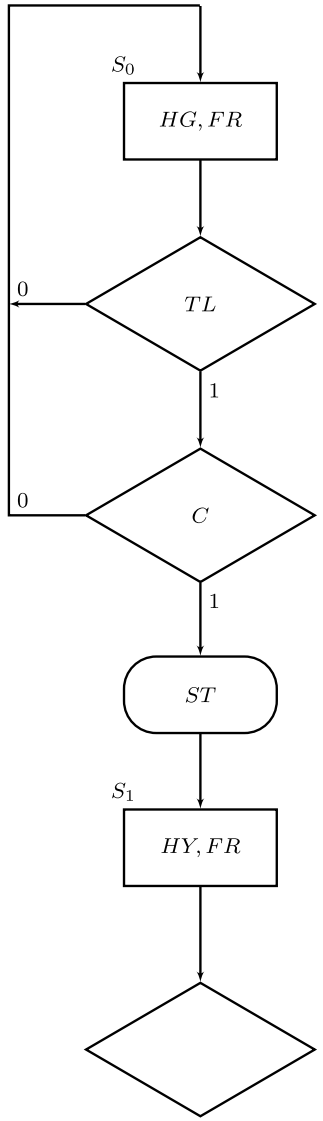
- A seguire si valutano le condizioni di transizione dei singoli stati
- Per S_0 serve considerare la presenza, o meno, di un veicolo in attesa sulla strada secondaria (C) a seguito di un periodo lungo di permanenza del verde sulla principale (TL)

Segnale di ingresso	Descrizione
RST	Porta la FSM allo stato iniziale
C	Presenza di veicolo su strada secondaria
TS	Fine intervallo breve
TL	Fine intervallo lungo

Segnale di uscita	Descrizione
HG, HY, HR	Attiva luce verde/gialla/rossa principale
FG, FY, FR	Attiva luce verde/gialla/rossa secondaria
ST	Inizio intervallo di tempo

Stato	Descrizione
S_0	Verde principale (Rosso secondaria)
S_1	Giallo principale (Rosso secondaria)
S_2	Verde secondaria (Rosso principale)
S_3	Giallo secondaria (Rosso principale)

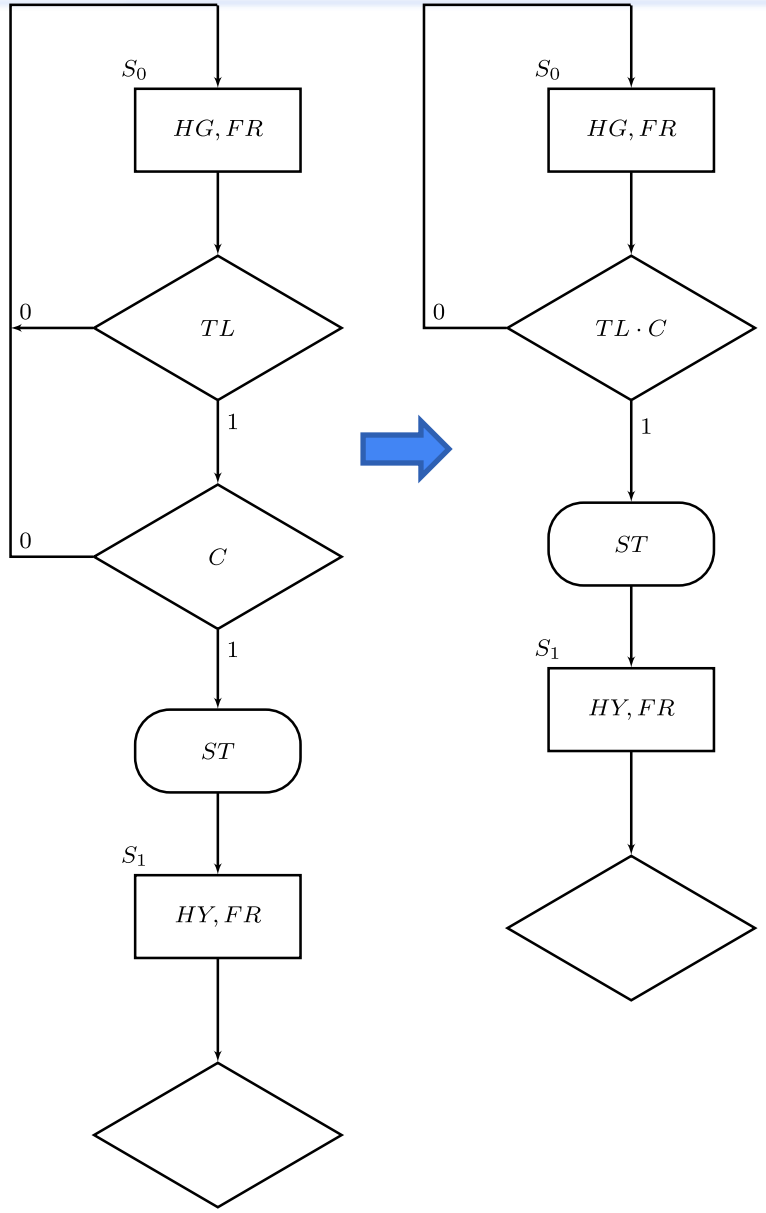




2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio ASM
Controllore semaforico

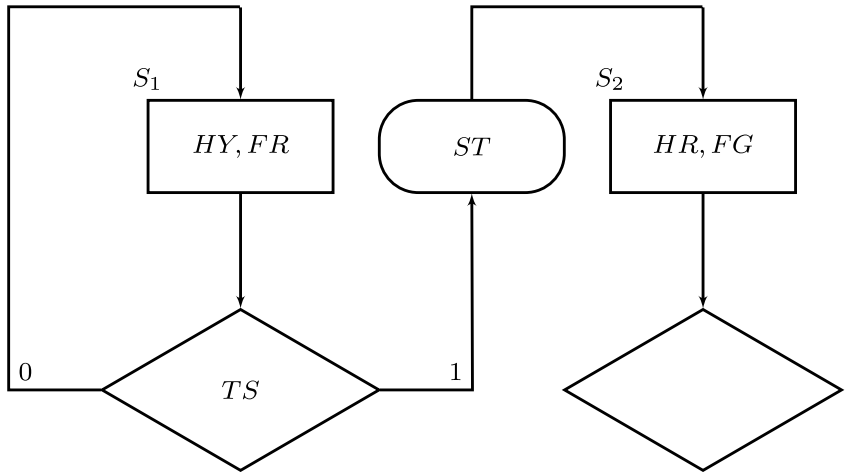
- A seguire si valutano le condizioni di transizione dei singoli stati
- Per S_0 serve considerare la presenza, o meno, di un veicolo in attesa sulla strada secondaria (C) a seguito di un periodo lungo di permanenza del verde sulla principale (TL)
- Le due condizioni possono essere aggregate in un singolo blocco di decisione, senza cambiare la natura del controllore
- Tutte le transizioni che coinvolgono un cambio di colore, sono temporizzare anche da un intervallo temporale corto avviato da ST per la permanenza in un certo stato



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio ASM
Controllore semaforico

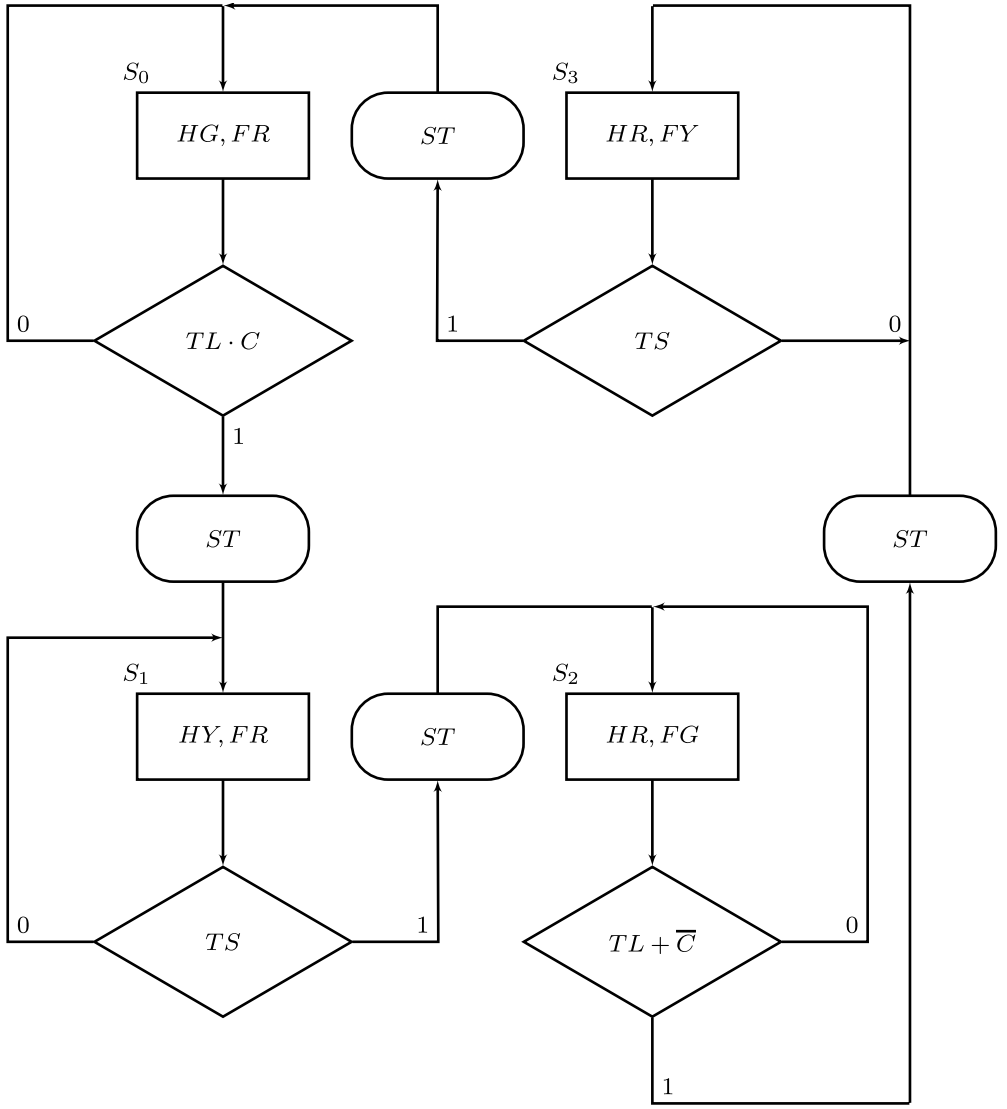
- In uscita da S_0 ma prima di entrare in S_1
  - Attivazione trigger conteggio intervallo corto
- Si rimane in S_1 fino alla fine del conteggio breve
  - Permane il *giallo sulla strada principale*
- In seguito si ha il cambio di colore
  - Permette il transito di veicoli sulla strada secondaria
- Il ragionamento si applica in *maniera speculare* per la coppia di stati S_3 ed S_0



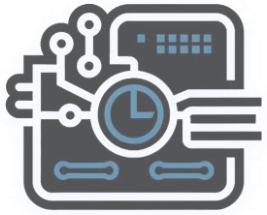
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio ASM
Controllore semaforico

- La condizione di uscita da S_2 è
  - O l'assenza di veicoli sulla careggiata
  - O la conclusione dell'intervallo lungo (durata del verde su una strada)
- Versione completa dell'ASM
  - Intera *sequenza di stati*
  - Tutte le *condizioni di transizione*
- Come per le FSM classiche, occorre fare attenzione a non dimenticare di modellare parti del comportamento atteso



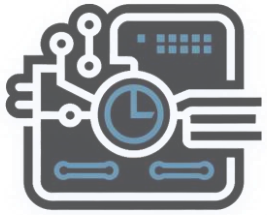
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio ASM
Serratura a combinazione numerica

- Sistema di riferimento
  - Una serratura seriale a 3 bit che controlla l'ingresso di una stanza
  - Ingressi: $RESET$, $ENTER$, un interruttore a 2 posizioni $KEY\text{-}IN$ per il bit di codice
  - L'uscita di sblocco $UNLOCK$ viene generata se il *codice introdotto è corretto*
  - Un segnale visivo di errore $ERROR$ si attiva se il *codice non è corretto*
  - La sequenza attesa di uso della serratura
    - (a) Premi $RESET$
    - (b) Introduci un bit di codice
    - (c) Premi $ENTER$
    - Ripeti (b) e (c) per altre 2 volte per completare la sequenza di input di tre cifre

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



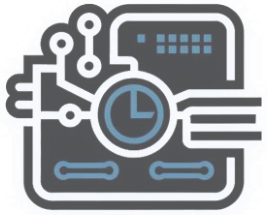
Esempio ASM
Serratura a combinazione numerica

- Sistema di riferimento
  - Una serratura seriale a 3 bit che controlla l'ingresso di una stanza
  - Ingressi: $RESET$, $ENTER$, un interruttore a 2 posizioni KEY_IN per il bit di codice
  - L'uscita di sblocco $UNLOCK$ viene generata se il *codice introdotto è corretto*
  - Un segnale visivo di errore $ERROR$ si attiva se il *codice non è corretto*
  - La sequenza attesa di uso della serratura
    - (a) Premi $RESET$
    - (b) Introduci un bit di codice
    - (c) Premi $ENTER$
    - Ripeti (b) e (c) per altre 2 volte per completare la sequenza di input di tre cifre

➔ La specifica è parziale

- Come viene definita la combinazione?
- Quando viene attivato $ERROR$ esattamente?

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



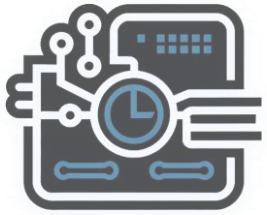
Esempio ASM
Serratura a combinazione numerica

- Sistema di riferimento
  - Una serratura seriale a 3 bit che controlla l'ingresso di una stanza
  - Ingressi: $RESET$, $ENTER$, un interruttore a 2 posizioni KEY_IN per il bit di codice
  - L'uscita di sblocco $UNLOCK$ viene generata se il *codice introdotto è corretto*
  - Un segnale visivo di errore $ERROR$ si attiva se il *codice non è corretto*
  - La sequenza attesa di uso della serratura
    - (a) Premi $RESET$
    - (b) Introduci un bit di codice
    - (c) Premi $ENTER$
    - Ripeti (b) e (c) per altre 2 volte per completare la sequenza di input di tre cifre

➔ La specifica è parziale
  - Come viene definita la combinazione?
  - Quando viene attivato $ERROR$ esattamente?

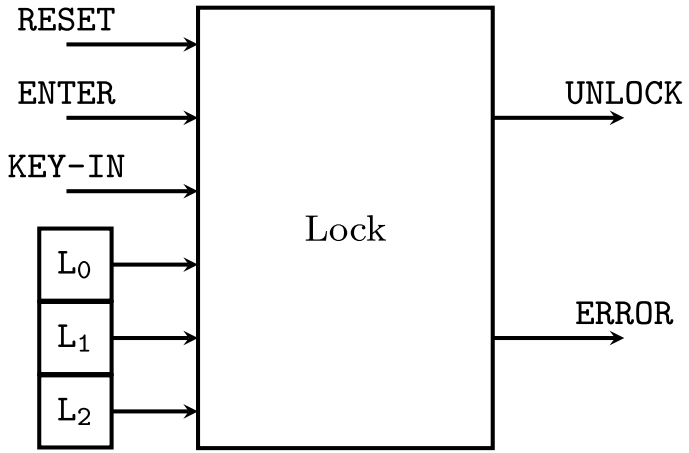
- Scelte di progetto
  - Combinazione memorizzata in un registro oppure «inclusa» nella logica di stato futuro
  - $ERROR$ attivato subito, appena identificato un errore, oppure alla fine del codice

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio ASM
Serratura a combinazione numerica

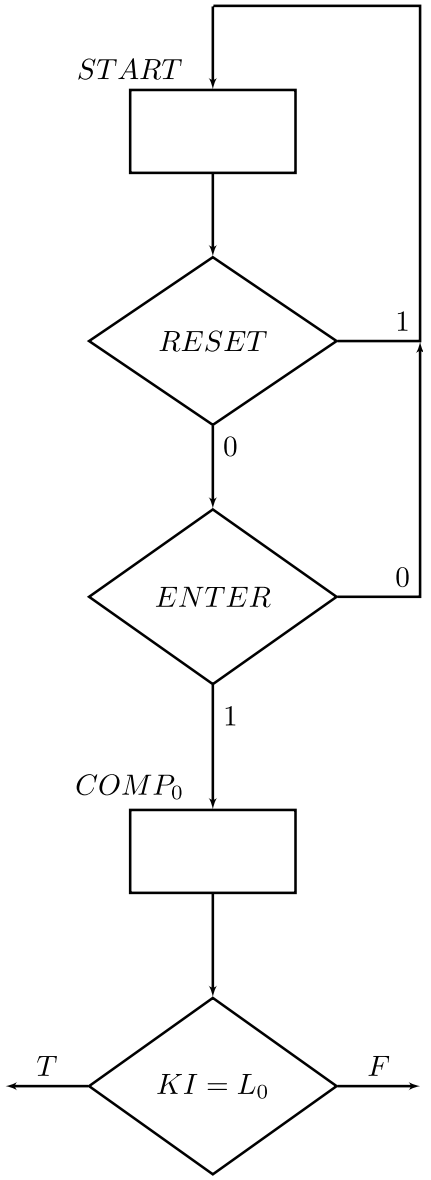
- Sintesi del problema
  - Schema a blocchi per *visualizzare le parti*
  - Ingressi/uscite per *l'interazione con l'ambiente*
  - Ingressi/uscite per mantenimento di informazioni interne
- Ingressi
  - *RESET*
  - *ENTER*
  - *KEY-IN*
- Uscite
  - *UNLOCK*
  - *ERROR*



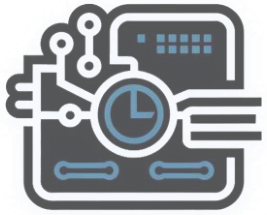
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio ASM
Serratura a combinazione numerica

- Enumerazioni degli stati
  - Quali sequenze portano all'apertura della serratura?
  - Quali sequenze portano a una condizione di errore?
- È ragionevole assumere uno *stato iniziale* ($START$) dove ancora *non si è fornito alcun input*
- Stati di confronto ($COMP_i$) per controllare gli input dell'operatore con i dati memorizzati nel registro interno
  - I confronti hanno successo se il bit in ingresso coincide con quello atteso per una certa posizione della combinazione

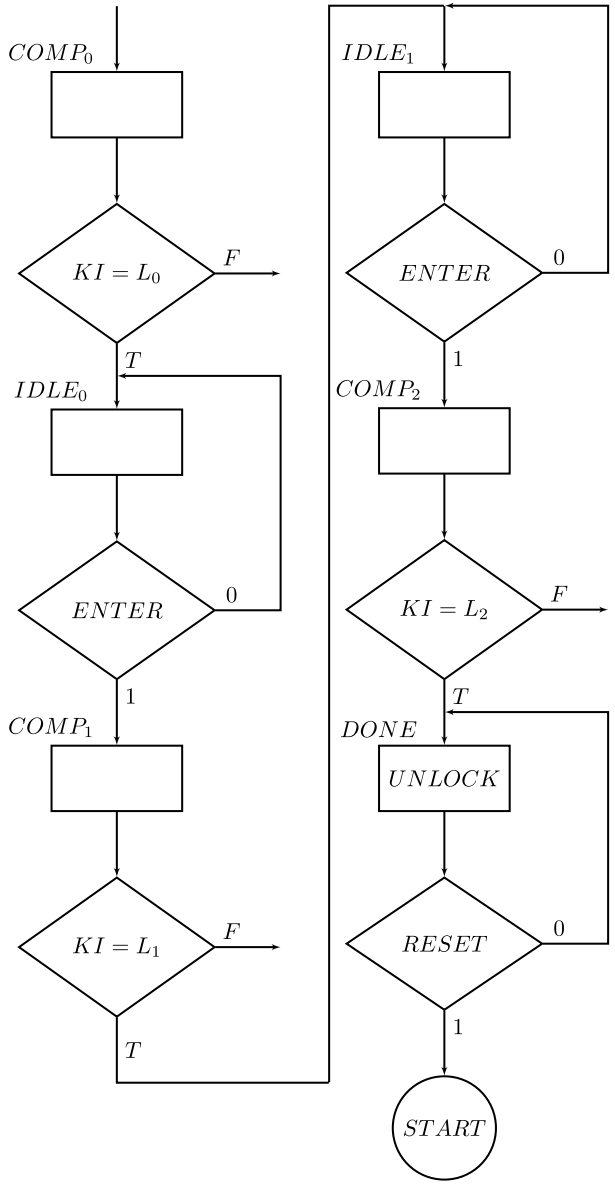


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio ASM
Serratura a combinazione numerica

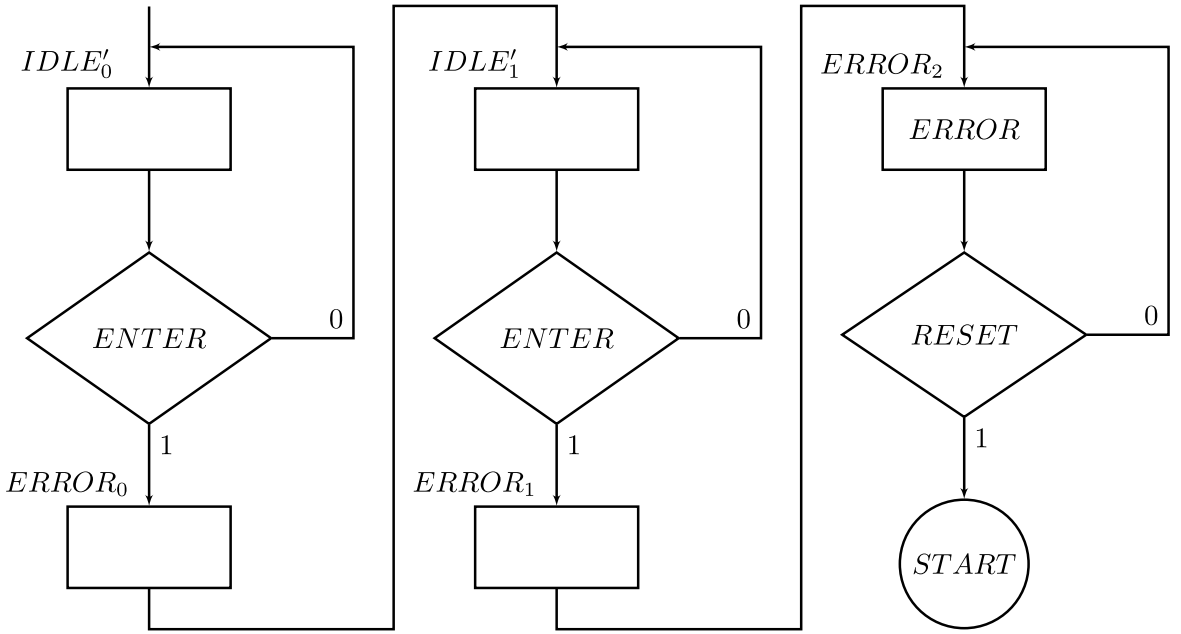
- Il *cammino di sblocco* passa (con successo) *tre stadi di confronto*, uno per ogni bit dell'operatore
- Stati di attesa ($IDLE_i$) tra *confronto* e *valutazione* del bit successivo
  - La macchina deve *attendere le tempistiche dell'operatore*
  - Si valuta il valore di $KEY{-}IN$ solo dopo la pressione di $ENTER$
- Condizioni di errore non gestite ancora
  - Lasciamo in sospeso alcuni «archi uscenti» dai blocchi
    - Ci ritorniamo in seguito…



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio ASM
Serratura a combinazione numerica

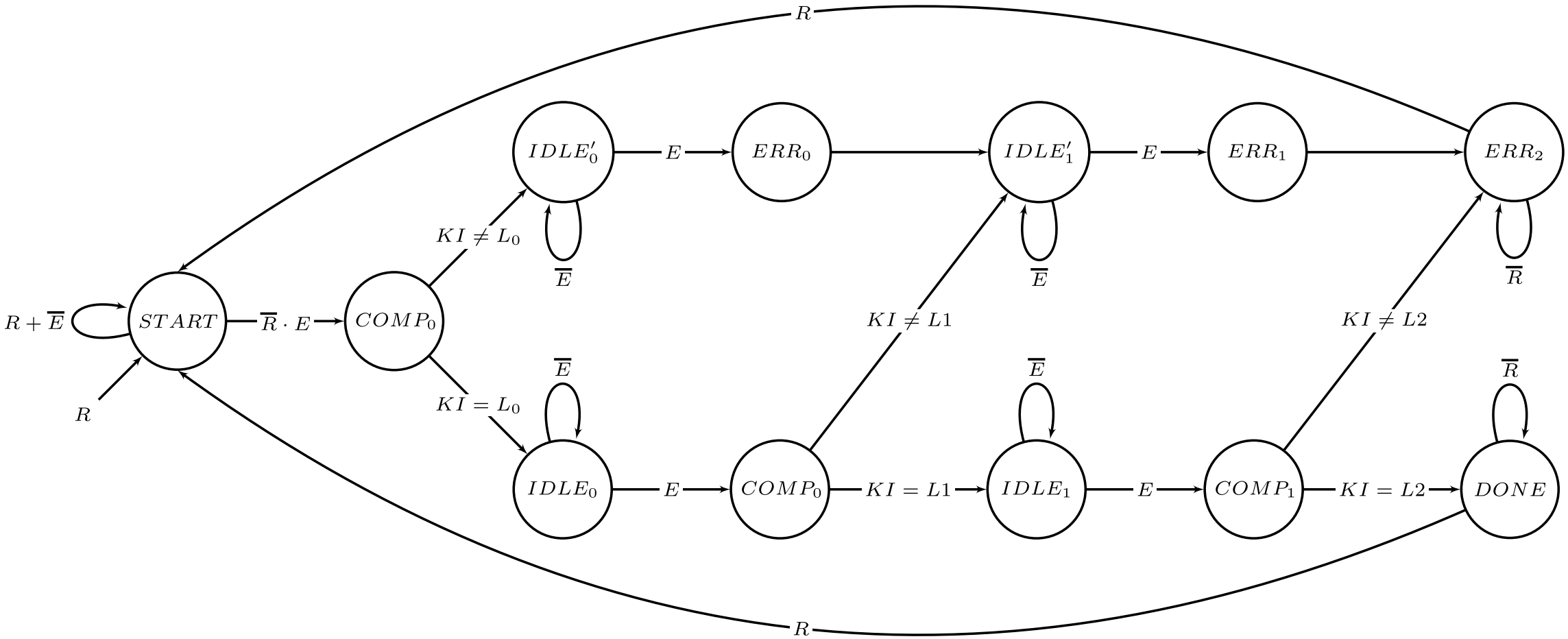
- Un *cammino alterato da errori* ha una struttura simile a quello di sblocco, ma porta a una *stato finale diverso* che attiva $ERROR$
  - Un *errore sul primo* o sul *secondo bit* instrada lungo il *cammino di errore*
    → Comportamento speculare a quanto visto prima
  - Un *errore sul terzo bit* conduce direttamente allo stato di segnalazione di errore



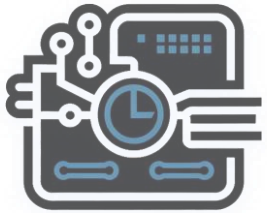
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio ASM
Serratura a combinazione numerica

- Dalle specifiche e dalla descrizione tramite ASM del sistema
  → Possiamo derivare la FSM adatta a modellarne il comportamento



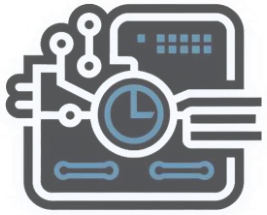
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Argomenti di approfondimento individuale

- A1) Tecniche automatiche per la codifica degli stati
- A2) Realizzazione di FSM con flip-flop JK
- A3) Decomposizione di FSM

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

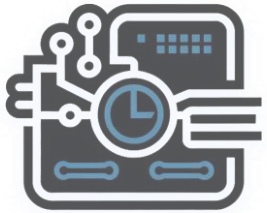


Codifica degli stati – Tecniche automatiche

A1

- NOVA: codifica degli stati per reti logiche a 2 livelli
- Vincoli di ingresso:
  - Stati con simile ingresso/uscita, codificati insieme
- Vincoli di uscita:
  - Stati che sono successori del medesimo predecessore codificati insieme
- Strategie:
  - *Greedy*: soddisfa più vincoli in ingresso possibili
  - *Hybrid*: soddisfa vincoli in ingresso in maniera più sofisticata
  - *I/O Hybrid*: soddisfa vincoli di ingresso e uscita
  - *Exact*: soddisfa tutti i vincoli di ingresso
  - *Input anneling*: come Hybrid con strategia di minimizzazione iterativa
  - *1-hot*: codifica one-hot
  - *Random*: usa una codifica casuale per gli stati

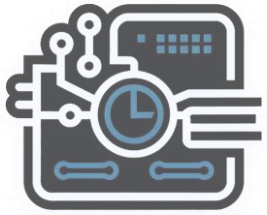
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Codifica degli stati – Tecniche automatiche

- Mustang: codifica degli stati ottimizzata per reti multi-livello
  - Punta alla riduzione dei letterali e non dei termini prodotto
  - Punta a massimizzare le sotto-espressioni comuni nelle funzioni di stato futuro
- Strategie:
  - *Sequential*: codifica gli stati sequenzialmente in maniera progressiva
  - *Fan-in*: stati con predecessori in comune ricevono codici vicini
  - *Fan-out*: stati con successori ed uscite in comune ricevono codici vicini
  - *One-hot*: codifica one-hot
  - *Random*: usa una codifica casuale per gli stati

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Scelta dei flip-flop

A2

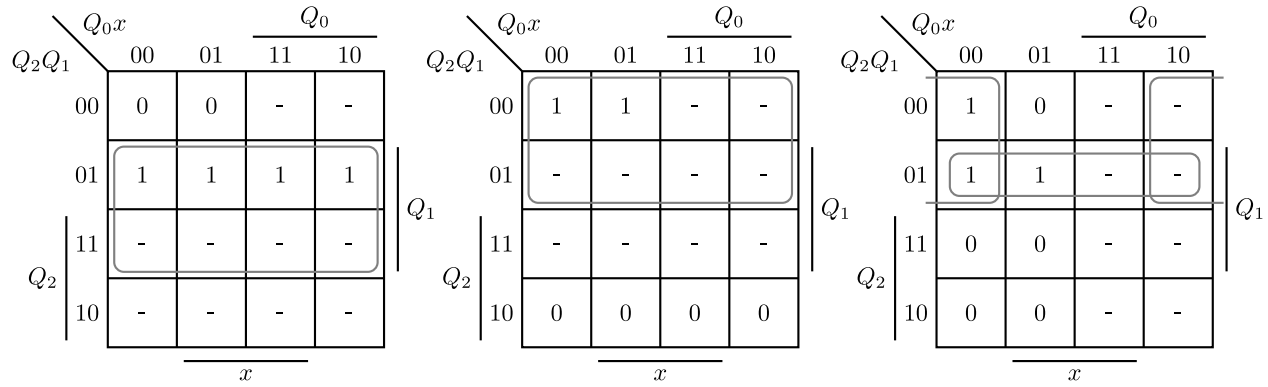
- Realizzazione con FF J-K
  - È necessario definire prima la tabella dello stato futuro trasformata

	Stato Futuro		J	K	J	K
Stato Presente	$x=0$	$x=1$	$x=0$	$x=0$	$x=1$	$x=1$
000	011	010	011	−−−	010	−−−
001	−−−	−−−	−−−	−−−	−−−	−−−
010	111	101	1−1	−0−	1−1	−1−
011	101	111	1−−	−10	1−−	−00
100	000	000	−00	1−−	−00	1−−
101	100	100	−0−	0−1	−0−	0−1
110	000	000	−−0	11−	−−0	11−
111	100	110	−−−	011	−−−	001

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Scelta dei flip-flop

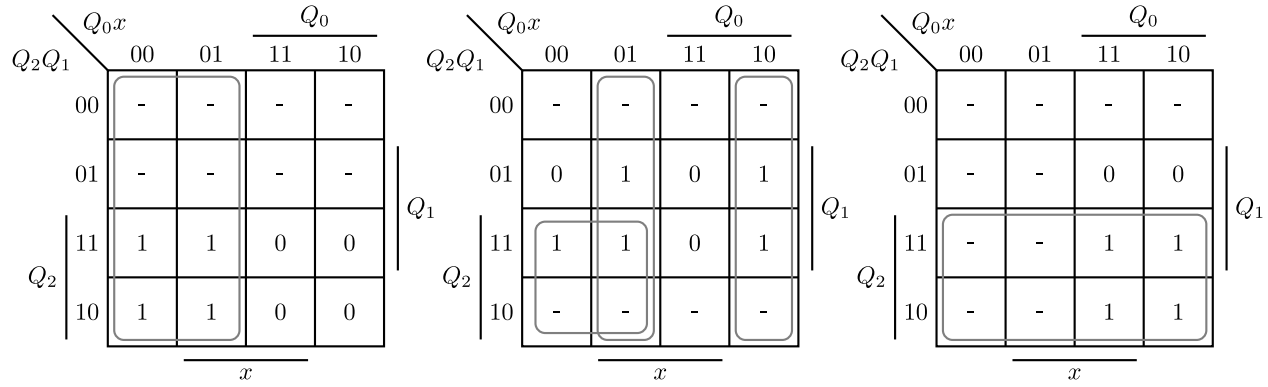
- Realizzazione con FF J-K
  - 9 termini prodotto e 14 letterali



$J_{Q_2} = Q_1$

$J_{Q_1} = \overline{Q_2}$

$J_{Q_0} = \overline{Q_2}Q_1 + \overline{Q_2}\overline{x}$



$K_{Q_2} = \overline{Q_0}$

$K_{Q_1} = \overline{Q_0}x + Q_0\overline{x} + Q_2\overline{Q_0}$

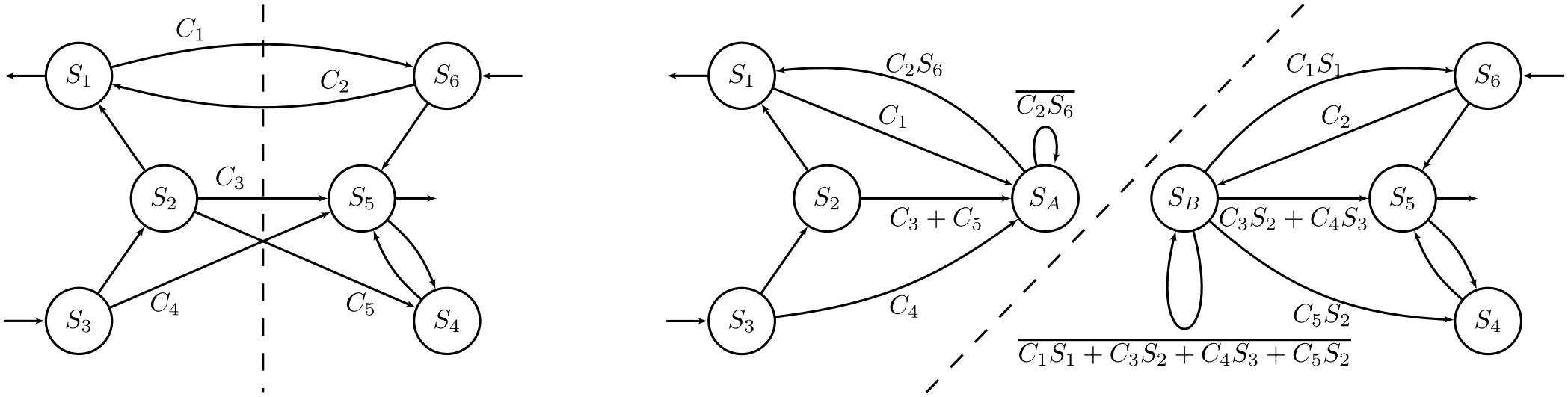
$K_{Q_0} = Q_2$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Decomposizione di FSM

A3

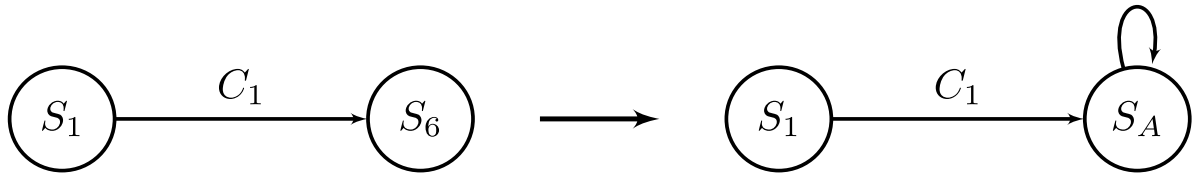
- Si identifica un taglio per partizionare la FSM e si introducono stati di attesa per «riscrivere» il comportamento del sistema, seguendo opportune regole



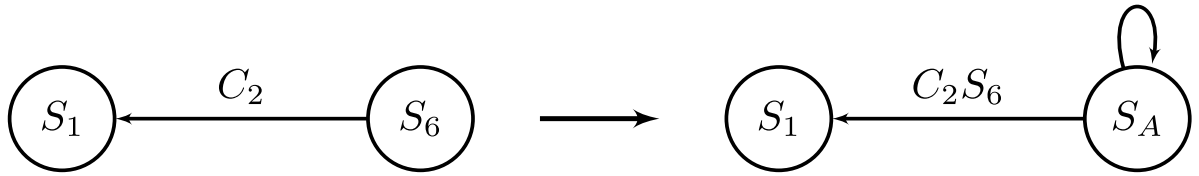
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Decomposizione di FSM

- Regola 1: trasformazione dello stato di partenza rispetto allo stato di attesa



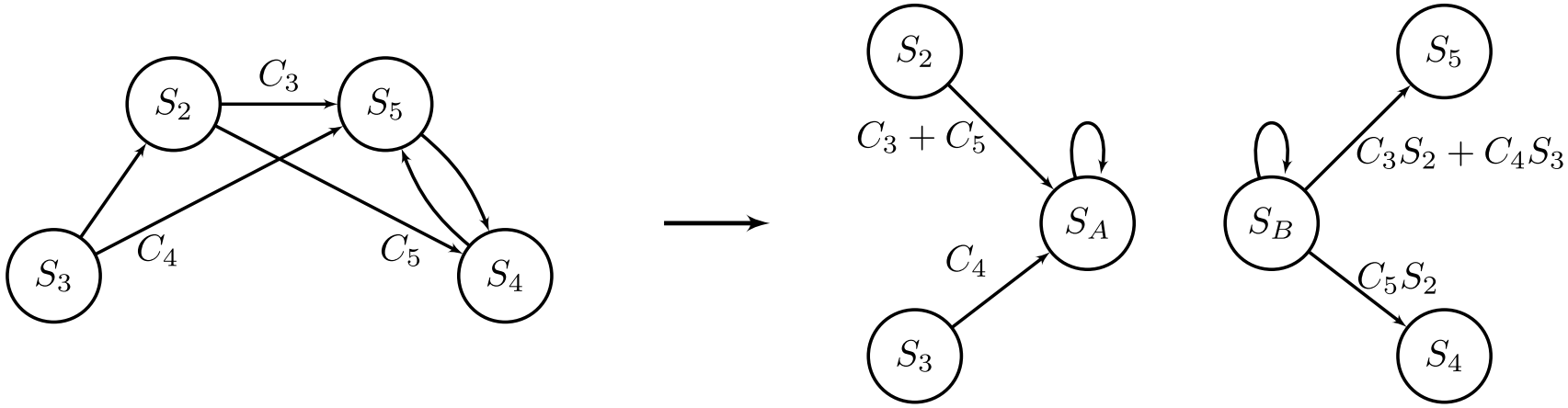
- Regola 2: trasformazione dello stato di arrivo



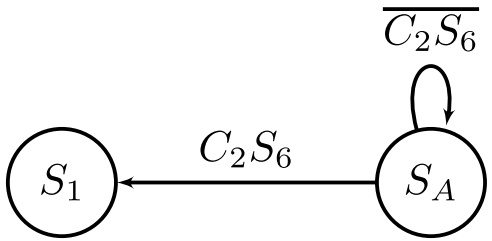
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Decomposizione di FSM

- Regola 3: transizioni multiple con stesso stato di partenza o arrivo



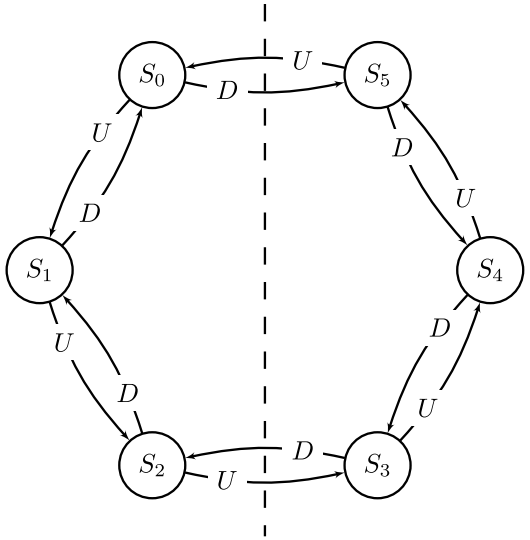
- Regola 4: condizione di permanenza in uno stato di attesa

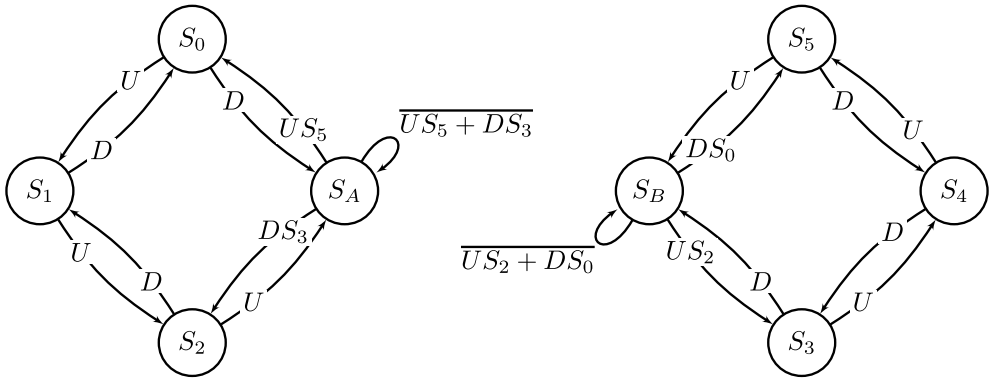


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Decomposizione di FSM

- Contesto di riferimento:
  - Contatore bidirezionale a 6 stati
  - Blocchi logici programmabili con 2 FF e logica combinatoria
- Decomposizione applicata:
  - Introduzione di due stati di attesa e partizionamento in due sotto-FSM da 4 stati ciascuna
- Sequenza di conteggio (de)crescente
  - Dopo S2 (S0) il sistema passa in SA, e lo lascia solo dopo S5 (S3)
  - Dopo S5 (S3) il sistema passa in SB, e lo lascia solo dopo S2 (S0)





2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini